



Module Code & Module Title

CC5009NI Cyber Security in Computing

Assessment Weightage & Type

60% Group Coursework 02

Year and Semester

2025/26 Autumn Semester

SN	Student Name	College ID	University ID
1	Poonam Maharjan	NP01NT4A240140	24046427
2	Bishu Kumar Shah	NP01NT4A240144	24046371
3	Manshi Bhandari	NP01NT4A240190	24046412
4	Sajeek Basnet	NP01NT4A240143	24046549

Assignment Due Date: Friday, May 15, 2026

Assignment Submission Date: Friday, May 15, 2026

Word Count (Where Required): 7260

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked.

I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Abstract

Web applications have become a critical component of modern digital infrastructure, yet they remain among the most frequently targeted systems by malicious actors. This report investigates injection flaws in web applications, with a focused examination of three prominent attack types: SQL Injection, OS Command Injection, and Server-side Template Injection (SSTI). Each of these vulnerabilities arises from a common root cause: the failure to properly validate and sanitize user-supplied input before it is processed by backend systems such as databases, operating system shells, and template rendering engines.

The background section establishes the theoretical foundation of injection attacks, drawing on established academic literature and industry frameworks including the OWASP Top 10 2025, which continues to rank injection flaws among the most critical web security risks globally. The demonstration section presents hands-on testing conducted entirely within controlled, isolated local environments using tools including XAMPP, PHP, MySQL, Python, Flask, and Kali Linux. SQL Injection was demonstrated against a deliberately vulnerable PHP login application, successfully bypassing authentication using crafted payloads. OS Command Injection was demonstrated through a Python Flask application that passed unsensitized input to the Linux shell via the subprocess module, allowing arbitrary system commands to be executed. SSTI was demonstrated through a Flask application using Jinja2's `render_template_string()` function with sanitized input, enabling server-side expression evaluation and internal configuration disclosure.

The mitigation section evaluates practical countermeasures for each attack, including input sanitization for SQL Injection, parameterized subprocess calls combined with strict IP validation for OS Command Injection, and safe template rendering practices for SSTI. Each mitigation was tested against the original payloads to verify effectiveness. The report concludes that injection vulnerabilities, while well understood, persist due to insecure development practices, and that a defense-in-depth approach combining input validation, parameterization, and secure coding standards is essential to protect modern web applications.

Table of Contents

1	Introduction	1
1.1	Types of Injection Flaws in Web Applications	2
1.2	SQL Injection	2
1.3	OS Command Injection	3
1.4	Server-Side Template Injection (SSTI)	3
1.5	CRLF/ HTTP Header Injection	4
1.6	NoSQL Injection	4
1.7	Technical Terminologies	5
1.8	Aim and Objectives	6
2	Background: The Landscape of Injection Flaws	7
2.1	Introduction to SQL Injection	7
2.1.1	How Injection Actually Works?	7
2.1.2	The Real-World Impact	8
2.2	Introduction on OS command Injection	9
2.2.1	How OS command Injection Works?	9
2.2.2	Real world Impact	9
2.3	Introduction on Server-side template Injection	10
2.3.1	How Server-side template Injection Works?	10
2.3.2	Real World Impact	10
3	Demonstration	11
3.1	Testing Environment Setup and Tools for SQL Injection	11
3.1.1	Creating the Database and Table	13
3.1.2	Inserting User Data	14
3.1.3	Developing the Vulnerable Login Page	15
3.1.4	Testing Normal Login Behavior	16
3.1.5	Performing SQL Injection Attack	17

3.2	Testing Environment Setup and Tools for OS Command Injection	18
3.2.1	Why a Virtual Environment was Used.....	19
3.2.2	The Vulnerable Application	20
3.2.3	Attack Payloads Used.....	21
3.3	Testing Environment Setup and Tools for Server-side Template Injection	22
3.3.1	Understanding the Jinja2 Template Engine	24
3.3.2	The Vulnerable Application	24
3.3.3	Attack Payloads Used.....	25
4	Mitigation.....	26
4.1	Test 1: SQL attack Input Sanitization.....	28
4.1.1	Step – 1 Temporarily Making it Vulnerable Again for Screenshots	28
4.1.2	Step – 2 Prepared Statements with Parameterized Queries.....	30
4.1.3	Step – 3 Valid Login (admin/ admin123).....	31
4.1.4	Step – 4 Injection Payload (' OR '1'='1/ pass123).....	32
4.2	Test 2: OS Command Injection	34
4.2.1	Step 1: Creation and Activation of Virtual Environment	34
4.2.2	Step 2: Installation of Flask in the Virtual Environment.....	35
4.2.3	Step 3: Creating the Vulnerable Python Application using nano command.	35
4.2.4	Step 4: Running the Vulnerable Server	36
4.2.5	Step 5: Running on the Firefox	36
4.2.6	Step 6: Checking Ping	37
4.2.7	Step 7: Trying Payload 1.....	38
4.2.8	Step 8: Trying Payload 2.....	39
4.2.9	Step 9: Trying Payload 3.....	39
4.2.10	Step 10: Mitigation Method (Removing Shell=True and Parameterizing the Command.)	40

4.2.11	Step 11: Implementation of Strict Input Validation which allows listing.	40
4.2.12	Step 12: Applying the Patched Code	41
4.2.13	Step 13: Testing Payload 1	41
4.2.14	Step 14: Testing Payload 2	42
4.2.15	Step 15: Testing Payload 3	42
4.3	Test 3: Server-Side Template Injection	43
4.3.1	Step 1: Installing Flask	43
4.3.2	Step 2: Vulnerable python code	44
4.3.3	Step 3: Run the python3 application and open the site	44
4.3.4	Step 4: Open Firefox	45
4.3.5	Step 5: Changing the URL and Seeing the Result	45
4.3.6	Step 6: Mitigation process	48
5	Evaluations	50
5.1	Evaluation of Input Sanitization	50
5.1.1	Advantages of Input Sanitization	50
5.1.2	Disadvantages Input Sanitization	51
5.1.3	Application Areas	52
5.2	Evaluation of OS Command Injection Mitigation	53
5.3	Command Parameterization.	53
5.4	Advantages of Command Parameterization (Avoiding shell=True)	53
5.5	Disadvantages of Command Parameterization (Avoiding shell=True)	53
5.5.1	Application Areas:	54
5.6	Strict Input Validation (Allow-listing / Type Checking)	55
5.7	Advantages:	55
5.8	Disadvantages:	55
5.9	Application Areas:	55

5.10	Principle of Least Privilege.....	56
5.11	Advantages:	56
5.12	Disadvantages:	56
5.13	Application Areas:	56
5.14	Evaluation of Server-Side Template Injection Mitigation Data Code Separation.....	57
5.15	Advantages:	57
5.16	Disadvantages:	58
5.17	Application Areas:	59
6	Conclusion	60
7	References.....	61
8	Appendix	65
8.1	Code for vulnerable Python Application	65
8.2	Patched Code	66

Table of Figures

Figure 1: Web Application Vulnerabilities	1
Figure 2: Types of Injection Flaws in Web Applications	2
Figure 3: SQL Injection workflow (PortSwigger, 2026).....	2
Figure 4:OS Command Attack Flow (Indusface, 2020)	3
Figure 5: SSTI (Demir, 2020)	3
Figure 6: NoSQL (Ansari, 2025).....	4
Figure 7: Mechanism for Injection (Geeks, 2025).....	7
Figure 8: XAMPP Control Panel running.	12
Figure 9: Database and users table created in phpMyAdmin.....	13
Figure 10 User data is inserted into the user's table.	14
Figure 11: Vulnerable login page created using PHP.....	15
Figure 12: Successful login using valid credentials.	16
Figure 13: SQL injection attack by passing login authentication.	17
Figure 14: Flask Installation	19
Figure 15 Flask Installation	22
Figure 16: CVSS Score	26
Figure 17: VS Code showing the vulnerable query.	28
Figure 18: Injection login working with admin/admin123.	29
Figure 19: Injection payload 'OR'1'=1	29
Figure 20: VS Code showing the prepared statement fixed code.	30
Figure 21: Normal login is still working after the fix.	31
Figure 22: Prepared Statement Used.....	32
Figure 23: Injection payload now blocked, showing Login Failed	32
Figure 24 Virtual Environment creation.	34
Figure 25 Activating Virtual Environment	34
Figure 26 Flask Installation	35
Figure 27 Creation of Vulnerable Python Application using nano command.....	35
Figure 28 Running the Vulnerable Server	36
Figure 29 Firefox opened.	36
Figure 30 Checking Ping	37
Figure 31 Result of Payload 1	38
Figure 32 Result of Payload 2	39

Figure 33 Result of Payload 3	39
Figure 34 Ping.....	41
Figure 35 Testing Payload 1 after applying mitigation method.....	41
Figure 36 Testing Payload 2 after applying mitigation method.....	42
Figure 37 Testing Payload 3 after applying mitigation method.....	42
Figure 38 Installing Flask	43
Figure 39 Vulnerable Python Code	44
Figure 40 Running python3.	44
Figure 41 Opening Link in Firefox	45
Figure 42 Viewing Result after Changing Result (1)	45
Figure 43 Viewing Result after Changing Result (2)	46
Figure 44 f-string in Code.....	46
Figure 45 Final Block of Code	47
Figure 46 Config Objects.....	47
Figure 47 Secure code.	48
Figure 48 Testing the Secure Code (1)	48
Figure 49 Testing the Secure Code (2)	49

Table of Tables

Table 1: Technical Terminologies	5
Table 2 System Configuration	11
Table 3 Table Structure	13
Table 4: Data Inserted	14
Table 5: Explanation of Payload	17
Table 6: OS Command Injection - System Configuration	18
Table 7: OS Command Injection Payloads and Their Intent	21
Table 8: SSTI System Configuration	23
Table 9: SSTI Payloads and Their Intent	25
Table 10: CVSS v3.1 Score	27
Table 11: Break down of each fixed component.	30
Table 12: Summary of Test 1	33

1 Introduction

Web applications have become an important part of the digital landscape, providing a quick approach to accessing information and allowing work, businesses, and fields to expand their capacity to reach more users.



Figure 1: Web Application Vulnerabilities

Frequently targeted and continuously attacked by threat actors on Web Applications due to their rapid production and broad attack surface, Web Applications are among the most vulnerable environments. The year-long increase in web application use has led to a rise in data and web traffic as well. For example, businesses migrated their system from offline to online, causing threat actors to exploit the system (Paul, et al., 2024).

The Open Worldwide Application Security Project (OWASP) has listed SQL Injection as one of the top five security risks on the latest 2025 list. As previously mentioned, it was one of the top three security risks (OWASP TOP 10, 2025).

1.1 Types of Injection Flaws in Web Applications

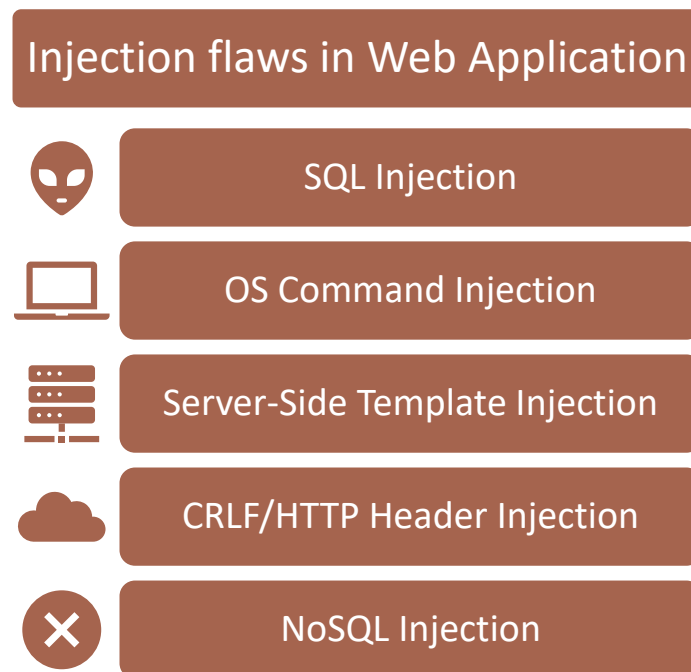


Figure 2: Types of Injection Flaws in Web Applications

1.2 SQL Injection

SQL injections are a technique that is executed by attackers, hackers, or threat actors to target the database and exploit vulnerabilities and flaws in web applications. It allows the attacker to breach confidentiality and integrity by accessing, modifying, or deleting the data using illegitimate commands and methods. (Nasareddin, et al., 2021)

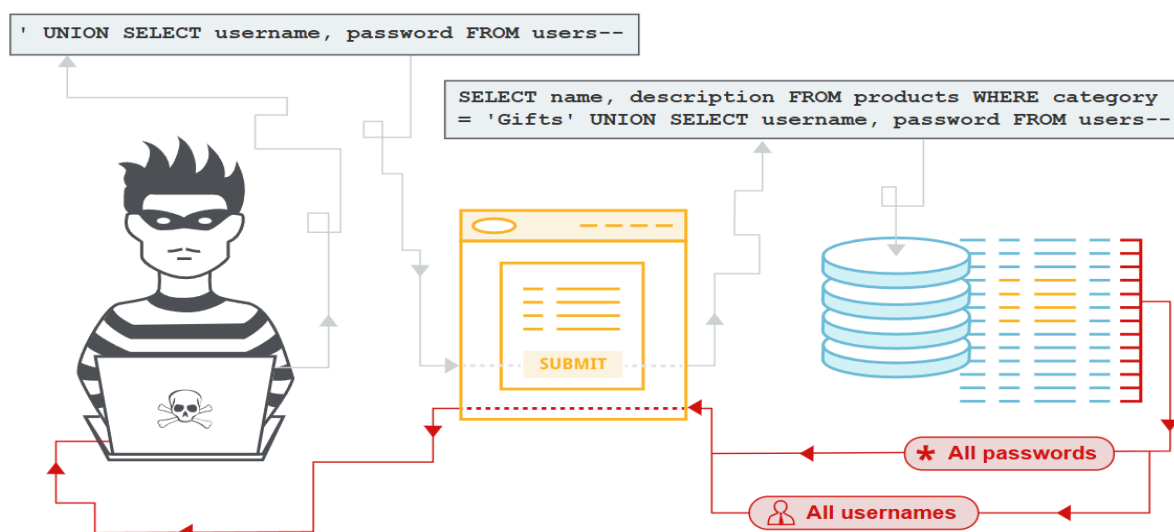


Figure 3: SQL Injection workflow (PortSwigger, 2026)

1.3 OS Command Injection

OS command injection is also understood as Shell injection. It allows an attacker to execute the OS commands on the server that is running an application, and it fully compromises the application and its data (portswigger, 2026)

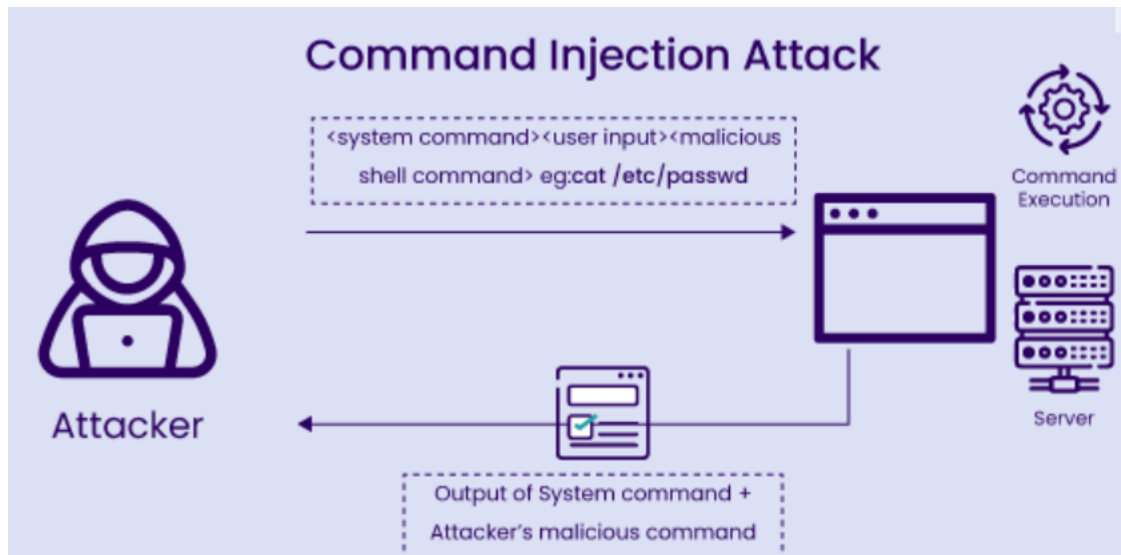


Figure 4:OS Command Attack Flow (Indusface, 2020)

1.4 Server-Side Template Injection (SSTI)

Server-Side Template Injection is understood as when a user-controlled input is added into a template, and then it is evaluated server-side by template engines like Twig, Velocity, and more. Unlike XSS, which executes in the browser, SSTI executes on the server-side, making it more dangerous.

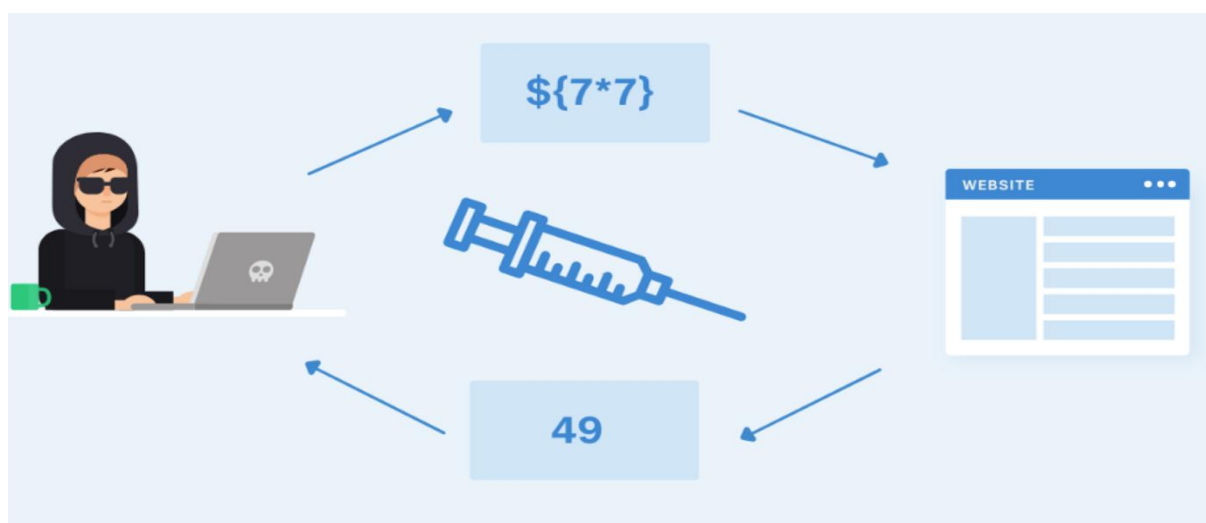


Figure 5: SSTI (Demir, 2020)

1.5 CRLF/ HTTP Header Injection

CRLF refers to Carriage Return Line Feed. It is an injection attack that occurs when a user manages to submit CRLF into an application, which is mostly done by modifying an HTTP Parameter or URL (owasp, 2026).

1.6 NoSQL Injection

NoSQL injection, it is a security weakness in a web application that uses a NoSQL database. NoSQL (Not Only SQL) refers to a database system that uses flexible data formats and that does not support SQL. Attackers can bypass authentication, get sensitive data, tamper with data on databases, and compromise the database. NoSQL injection vulnerabilities occur as the developer processes the code without properly sanitizing it (imperva, 2026).

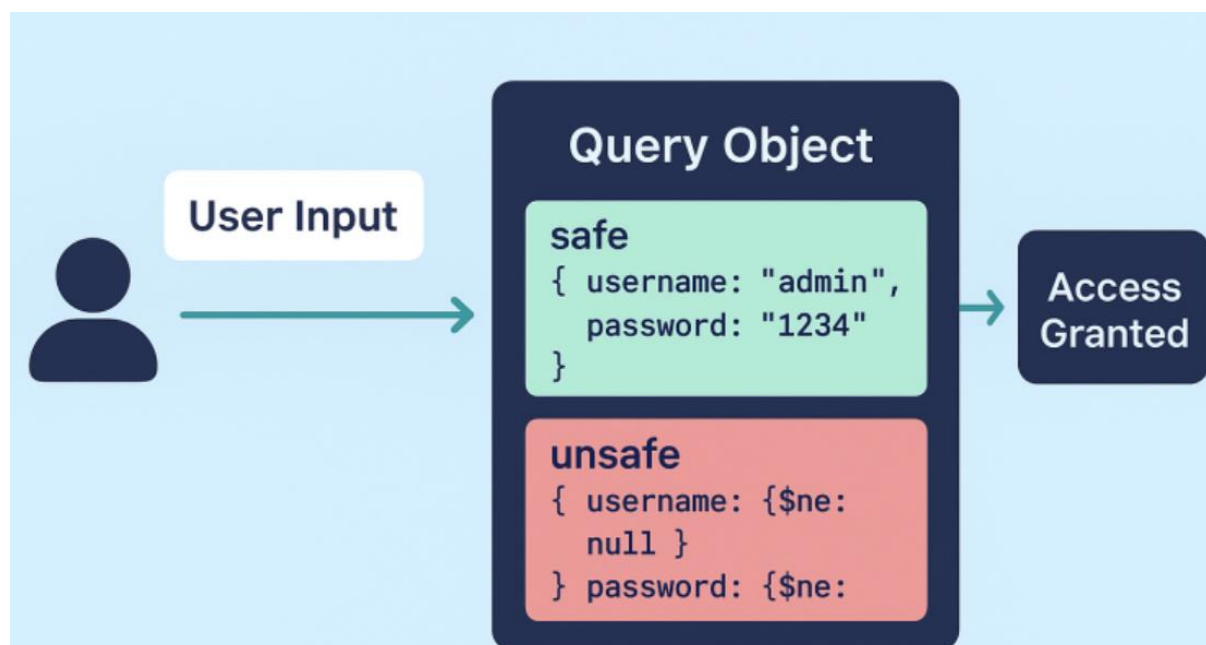


Figure 6: NoSQL (Ansari, 2025)

1.7 Technical Terminologies

Technical Terms	Definitions
Confidentiality	It is defined as the process of keeping sensitive information secret and confidential. If the confidential fails, the data is leaked or it is exposed.
Integrity	Integrity means, the data stays unchanged and stays as it is until and unless the authorized user changes. If unauthorized user access and changes, modifies or tempers, the integrity fails.
Attack Surface	Attack surface is understood as all the possible vectors or ways an attacker can attack and try to break into the system. If there is more attack surface in the system, it has more risk for the system.
OWASP Top 10	It is the list of most critical web security risks listed by the OWASP, which is a non-profitable organization.
Data Exfiltration	The process of stealing data from a system, computer or by various cyberattack methods is known as Data Exfiltration.
Flask	Flask is a lightweight web framework which is used to build web application (Kartik, 2026).
Jinja	Jinja is a reading engine for python programming language which read text, it is one of the popular and straightforward syntax (Abhijeet, 2025).

Table 1: Technical Terminologies

1.8 Aim and Objectives

This coursework aims to research Injection flaws in web applications, and understand their background, how they can be exploited, their mitigation as well as how those exploited vulnerabilities can be mitigated, and their application areas.

Objectives

The objective of this coursework is as follows:

- Research on injection flaws in web applications.
- Analyzing the several types of injection attacks.
- Understanding the background of injection flaws in web applications.
- Demonstrating an attack in a controlled and safe environment.
- Testing different scenarios to mitigate the issue or flaws.
- Providing the advantages and disadvantages of the mitigation technique.
- Understanding the application areas.

2 Background: The Landscape of Injection Flaws

2.1 Introduction to SQL Injection

Structured Query Language (SQL) is a code injection technique which is used to change or modify data from SQL database. By injecting specialized SQL commands into the input field or entry field, where an attacker can execute commands that allow the retrieval of data from the database. (cloudflare, 2026)

2.1.1 How Injection Actually Works?

Injection happens when an application takes input from a user but fails to properly verify or sanitize, the entered input, before passing it along to the backend and providing the access (Halfond & Orso, 2020).

In a secure app, the input field is sanitized and verified so that anything the user types into are checked. But instead of a secure app, if the app is vulnerable, the system gets confused between malicious or harmless command, which allows the attacker to gain advantage of the vulnerable application and slip the malicious code which is disguised as a normal input, and the backend which is a database engine, it reads the command and gets tricked into running the attacker's malicious code. (Hoang Viet Long, 2020)

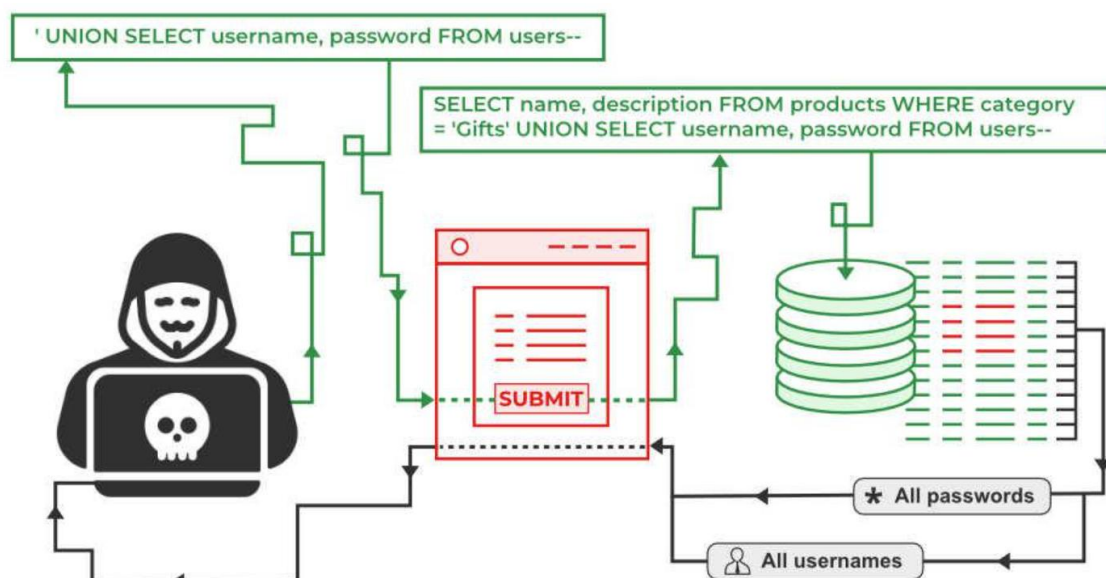


Figure 7: Mechanism for Injection (Geeks, 2025)

2.1.2 The Real-World Impact

The real-world impact after the successful injection attack is followed by, leaking extremely sensitive data, data getting stolen, modified or destroyed (Hoang Viet Long, 2020). It results in massive data breach, as attacker bypass authentication with the aim to steal, expose or sell the confidential data and information like passwords, health records, credit card numbers of the users.

There are chances of infrastructure getting compromise as SQL injection can execute commands on the underlying servers, allowing attackers to gain full access and control of the system.

2.2 Introduction on OS command Injection

OS command injection is one of the web application vulnerabilities, which allows the attacker to execute commands on the underlying operating system, as these vulnerabilities occur when operating system commands call web application with user-supplied input, provided as an argument (Mathur, 2025).

OS command injection vulnerabilities appear when the developers fail to properly validate or sanitize the user input field when constructing commands to execute on underlying OS (Cybersecurity & Infrastructure Security Agency, 2024).

2.2.1 How OS command Injection Works?

The OS command injection works, using user supplied input (like ping, dir, ls). If the input is not validated, then the attacker injects extra command into it to exploit.

The attacker uses special shell characters to add malicious command with a legit one.

For Example, there is a vulnerable app which takes an IP address and runs the command **ping -n 3 <input>**. An attacker submits this command.

8.8.8.8 & whoami

The system then executes both: ping -n 3 8.8.8.8 and whoami, exposing the system information.

There is blind injection as well, which shows no visible outputs, as attacker instead use time delays (ping -c 25) or sends data to an external server which attacker controls to confirm the attack works. (Nidecki, 2026)

2.2.2 Real world Impact

The real-world impact after accessing the system, the attackers can gain unauthorized access to the system, escalating the privileges from a stander users to a root administrator level, As the attacker have completely control over the compromised system, and also the ability to modify, delete or execute any files.

Malware and Ransomware installation in the system, is also an impact with the command execution capability, attackers can download or install malicious software which encrypts the victims' file and later asks for Ransome, the installed malware can also spread to other connected system within the same network, damaging and slowly corrupting overall network.

2.3 Introduction on Server-side template Injection

Server-side template injection vulnerabilities (SSTI), occurs when input is injected into a template which is an unsafe manner and results in remote code execution on the server (owasp, 2026).

2.3.1 How Server-side template Injection Works?

Server-side template works by exploiting the way templates processes user inputs, as when the template engine receives any input form user, it merges the input with a predefined template to generate dynamic content.

Attackers first identify the template engine which is used, and once it is found attacker crats specific payload which template engine will execute.

For example, in a Jinja2 template, an attacker can inject `{{7*7}}`, to perform arithmetic operation or even more complex commands. Attackers inject payloads are processed by server-side template engine which interprets them as part of template, which allows attackers to execute arbitrary code, manipulate data, or perform other malicious actions. (Twingate Team, 2024)

2.3.2 Real World Impact

Server-side template allows remote code execution because the code process is on the server, and attackers can pass specific payloads like in python, Jinga2 which appeal to system (Mahesh, 2026). STTI is chained with other vulnerability to multiply the damage level, like Server-Side Request Forgery (SSRF), where attacker uses remote code execution on the template to force the server to make unauthorized requests to the internal system, extracting important credentials. Cross-site Scripting (XSS) can also be used to multiply the damage as the attacker can inject malicious code into client-side, which is then executed in the browser of users visiting the compromised websites.

3 Demonstration

3.1 Testing Environment Setup and Tools for SQL Injection

The SQL injection vulnerability was evaluated within a local controlled environment using XAMPP, which offers the Apache web server and MySQL database that are needed to execute the web app. The system has been developed in-house to make sure that all testing operations have been done safely without any interference with the outer networks and real users.

Component	Description
Server Platform	XAMPP
Programming Language	PHP
Database	MySQL
Database Name	Dragon_db
Table Name	Users
Operating System	Windows
Development Tool	Visual Studio Code

Table 2 System Configuration

The vulnerable login software was developed on PHP and MySQL to model a real-world authentication system. The application lets users type in a username and password, which are matched to the database records. Nevertheless, the login system was deliberately developed without adequate input validation to show how SQL injection attacks can be done.

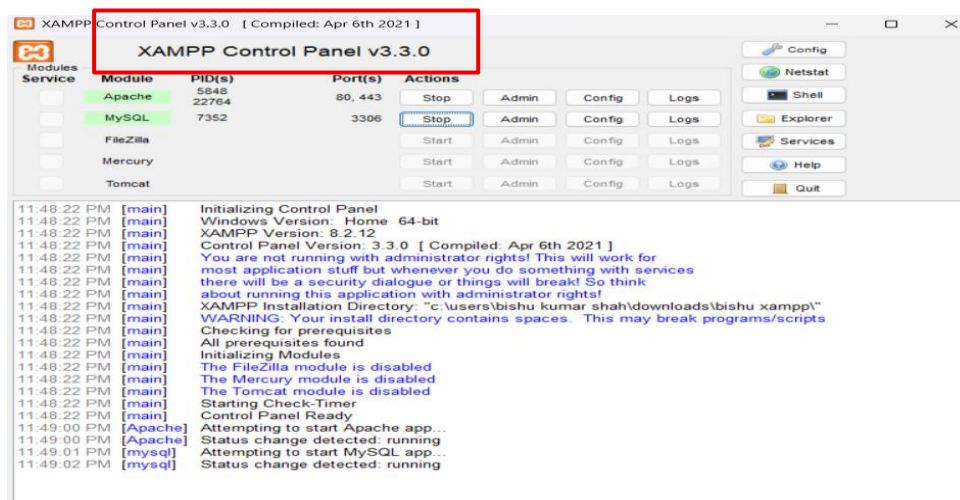


Figure 8: XAMPP Control Panel running.

3.1.1 Creating the Database and Table

The demonstration was initiated by developing a database and table that would hold user credentials. This was done through the phpMyAdmin interface that comes with XAMPP.

Field	Type	Description
Id	INT	Unique user identifier
Username	VARCHAR	Stores username
Password	VARCHAR	Stores password

Table 3 Table Structure

A new database was created, which is called `dragon_db`, and then a table was created with three fields, which are `id`, `username`, and `password`. The ID field was defined as a primary key and auto incremented to give a new value with every new record.

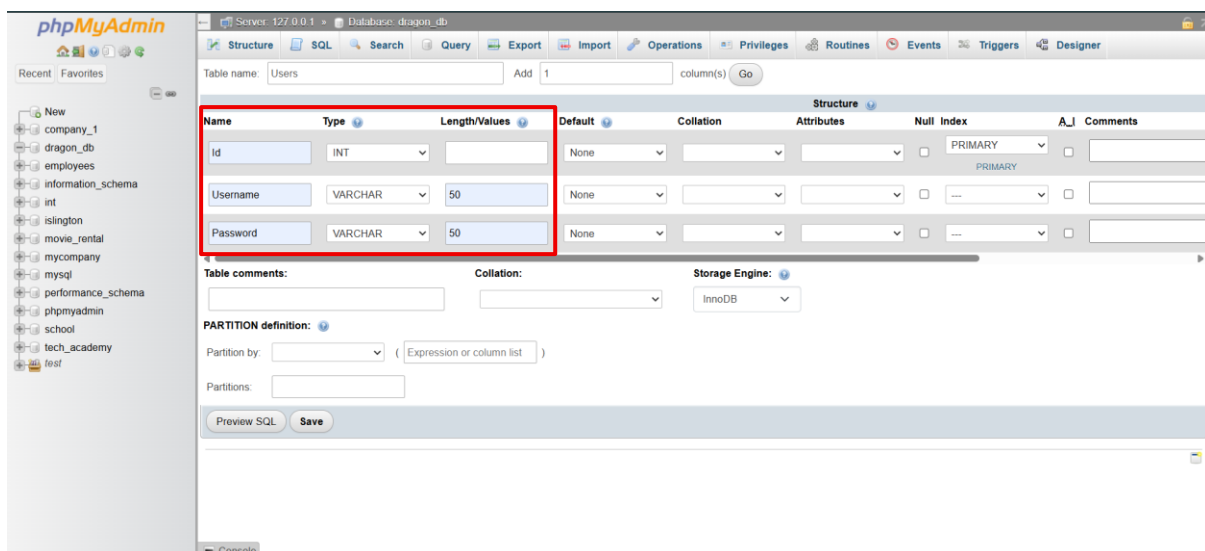


Figure 9: Database and users table created in phpMyAdmin.

This structure represents a simple user authentication database commonly used in web applications.

3.1.2 Inserting User Data

After creating the table, sample user records were inserted into the database. These records were used to test the login functionality and demonstrate how SQL injections can manipulate database queries.

Username	Password
admin	admin123
user1	pass123

Table 4: Data Inserted

These records simulate normal user accounts in a web application.

The screenshot shows the phpMyAdmin interface for a database named 'dragon_db'. The 'users' table is selected, and the 'Insert' tab is active. The table structure is shown at the top with columns: Id (int(11)), Username (varchar(50)), and Password (varchar(50)). Below the structure, the 'Insert' form is displayed, which is highlighted with a red box. The form has two rows of input fields. The first row has 'admin' entered in the Username field and 'admin123' in the Password field. The second row has 'user1' entered in the Username field and 'pass123' in the Password field. There are 'Go' buttons next to each row of input fields. At the bottom of the form, there are options to 'Insert as new row' and 'and then Go back to previous page', along with 'Preview SQL', 'Reset', and 'Go' buttons.

Figure 10 User data is inserted into the user's table.

3.1.3 Developing the Vulnerable Login Page

A login page named **login.php** was created using PHP. This page connects to the database and checks whether the entered username and password match any record on the user's table.

The following SQL query was used to validate user credentials:

```
SELECT * FROM users WHERE username='$username' AND password='$password'.
```

This query directly includes user input without validation or parameterization. As a result, it is vulnerable to SQL injection attacks.

When a user enters login credentials, the application sends the input directly to the database for verification.

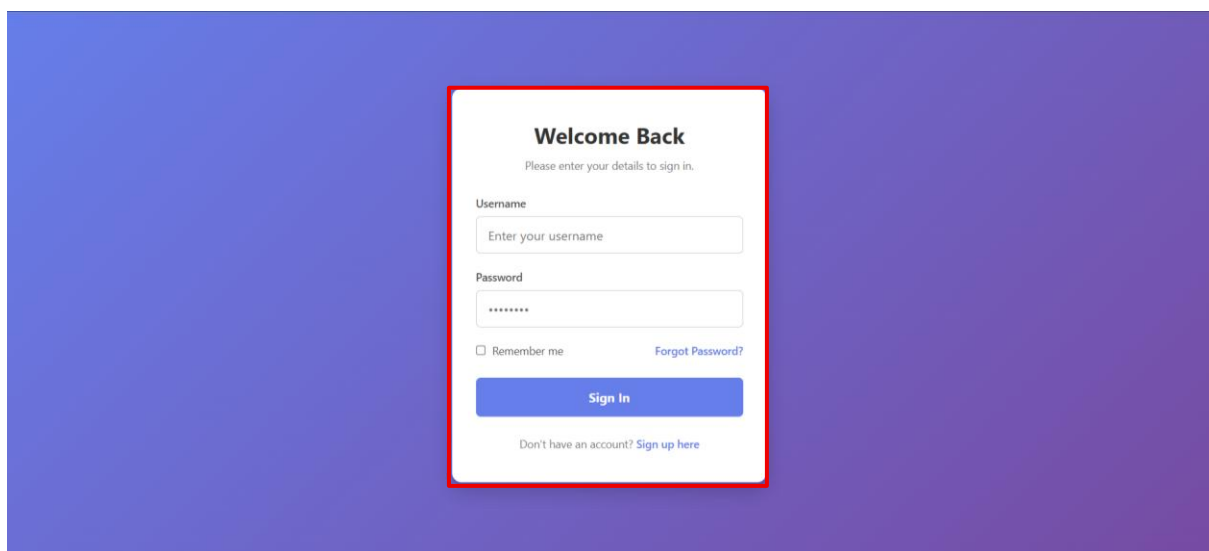


Figure 11: Vulnerable login page created using PHP.

3.1.4 Testing Normal Login Behavior

Before performing the SQL injection attack, the login system was tested using valid credentials to confirm that it was functioning correctly.

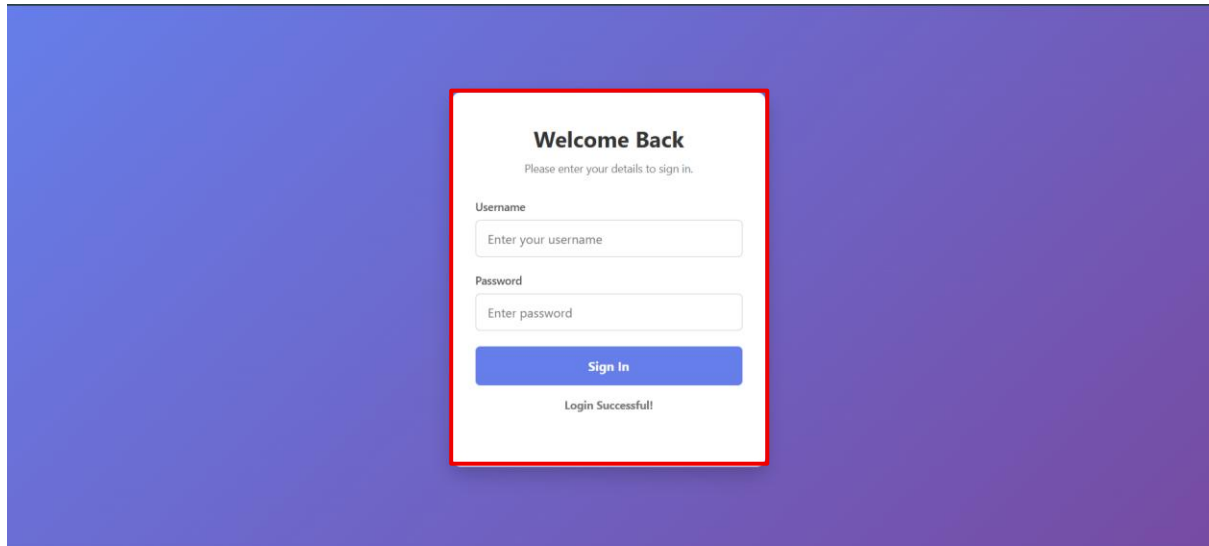


Figure 12: Successful login using valid credentials.

This confirmed that the login system was correctly retrieving user data from the database.

3.1.5 Performing SQL Injection Attack

After confirming normal login behavior, a SQL injection payload was entered into the login form to bypass authentication.

Payload Used

Username: **OR '1'='1**

Password: **pass123**

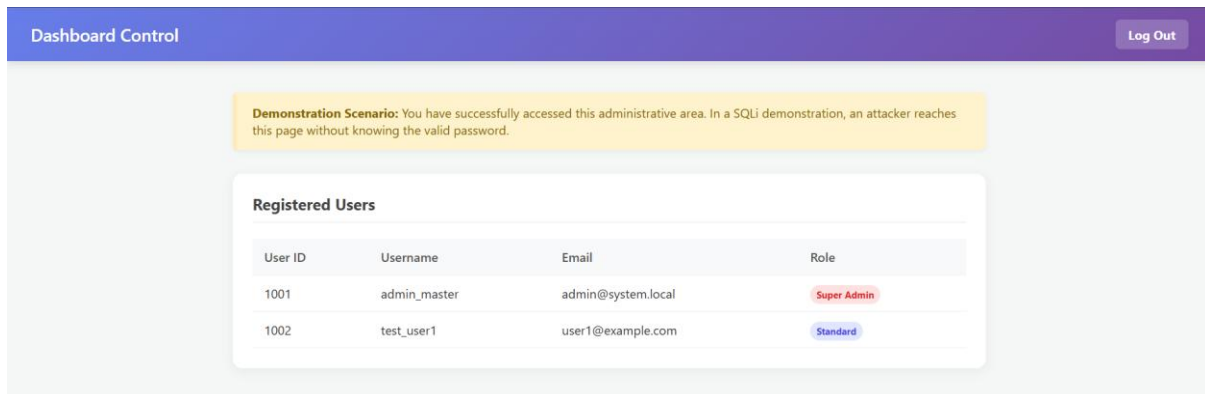


Figure 13: SQL injection attack by passing login authentication.

Component	Meaning
'	Close the original SQL string
OR	Logical operator
'1'='1	Always true condition

Table 5: Explanation of Payload

3.2 Testing Environment Setup and Tools for OS Command Injection

OS Command Injection was demonstrated within a controlled and isolated local environment running on Kali Linux. A deliberately vulnerable web application was built from scratch using Python and the Flask micro-framework, which simulates a real-world network diagnostic tool that accepts user-supplied IP addresses and passes them directly to a system-level ping command. No external networks, live servers, or real user data were involved at any point during the testing process, ensuring the demonstration remained entirely safe and ethical.

Component	Description
Operating System	Kali Linux
Programming Language	Python 3
Web Framework	Flask (installed via pip in a virtual environment)
Virtual Environment	venv (Python built-in module)
System Module Used	subprocess (Python standard library)
Vulnerability Introduced	shell=True with unsensitized string concatenation
Browser for Testing	Mozilla Firefox
Application Port	http://127.0.0.1:5000

Table 6: OS Command Injection - System Configuration

3.2.1 Why a Virtual Environment was Used.

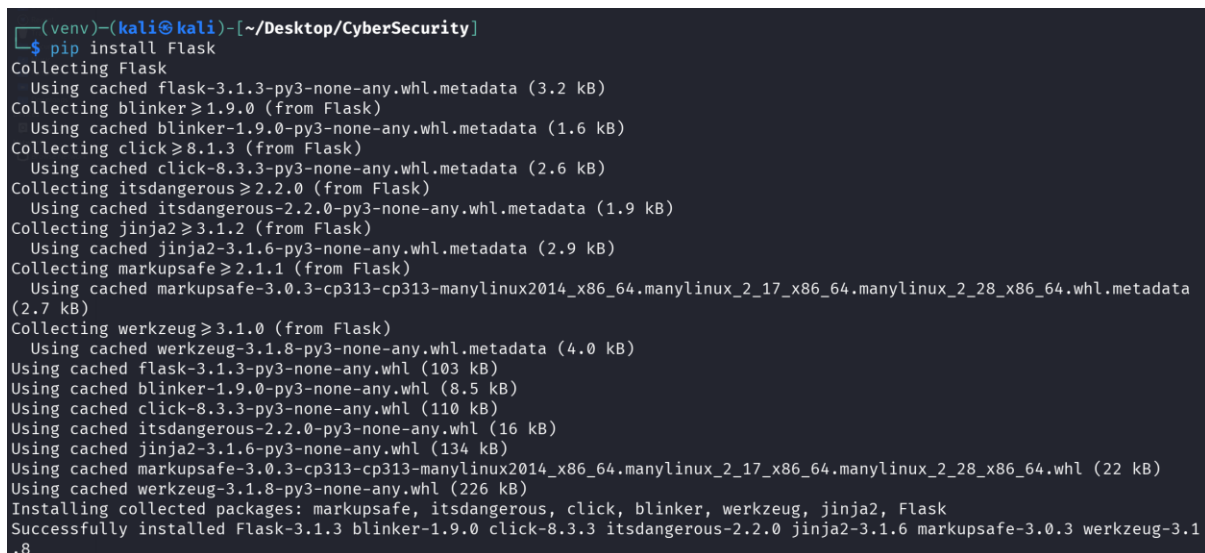
A Python virtual environment (venv) was created before installing Flask. This is standard practice in Python development and ensures that the Flask installation and its dependencies remain isolated from Kali Linux's system-level Python packages. Mixing pip-installed packages with system packages on Kali Linux can cause conflicts or brake system tools, so the virtual environment prevents this entirely. The environment was created and activated using the following terminal commands:

```
python3 -m venv venv
```

```
source venv/bin/activate.
```

Once activated, Flask was installed inside the virtual environment using:

pip install Flask



```
(venv)-(kali@kali)-[~/Desktop/CyberSecurity]
$ pip install Flask
Collecting Flask
  Using cached flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from Flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from Flask)
  Using cached click-8.3.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from Flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from Flask)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from Flask)
  Using cached markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from Flask)
  Using cached werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached click-8.3.3-py3-none-any.whl (110 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, Flask
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.3.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8
```

Figure 14: Flask Installation

3.2.2 The Vulnerable Application

The vulnerable application was created using the nano text editor directly in the terminal. It simulates a network diagnostic tool; a type of utility commonly found in web-based server management dashboards and router administration panels. The tool accepts an IP address from the user through an HTML form and passes it to the Linux ping command via Python's subprocess module.

The core vulnerability exists in the following line of the application's source code:

```
command = "ping -c 1" + user_input  
  
output = subprocess.check_output (command, shell=True, text=True,  
                                stderr=subprocess.STDOUT)
```

Here, the `user_input` variable is obtained directly from the HTTP POST request without any validation, filtering, or sanitization. When `shell=True` is set, Python passes the entire command string to the operating system's shell interpreter (`/bin/sh` on Linux). Because of this, an attacker can append shell metacharacters such as semicolons (`;`) or double ampersands (`&&`) to chain additional OS commands onto the ping instruction. The shell interpreter executes all of them in sequence, giving the attacker arbitrary command execution on the server.

The full source code of this vulnerable application is provided in [Appendix 7.1](#).

3.2.3 Attack Payloads Used

There are three injection payloads prepared before testing, each designed to demonstrate a different capability that an attacker could achieve through OS Command Injection:

S.N.	Payload	Intended Outcome
1	127.0.0.1; cat/etc/password	Read the system's user account file, which demonstrates sensitive file disclosure
2	127.0.0.1; ls -la	List all files in the application's working directory, which demonstrates file system access
3	127.0.0.1 && whoami	Reveal the operating system user running the web server, demonstrating privilege identification

Table 7: OS Command Injection Payloads and Their Intent

Each payload begins with a valid IP address (127.0.0.1) to satisfy the first of the ping command. The shell metacharacter that follows (either ; or &&) then introduces a second, malicious command. Because shell=True allows the shell to interpret these characters, both commands are executed, and their combined output is returned directly to the web browser. The detailed results of each payload test are presented in Section 4.2.

3.3 Testing Environment Setup and Tools for Server-side Template Injection

Server-Side Template Injection (SSTI) was demonstrated using another locally developed Flask application running on the same Kali Linux machine. Unlike OS Command Injection, where the attacker abuses a system-level call, SSTI exploits the template rendering engine that a web framework uses to generate dynamic HTML responses. In this case, the vulnerable component is Flask's built-in Jinja2 template engine, which was given direct access to raw, unsensitized user input through Python's `render_template_string()` function.

```
(venv)-(kali@kali)-[~/Desktop/CyberSecurity]
$ pip install Flask
Collecting Flask
  Using cached flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from Flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from Flask)
  Using cached click-8.3.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from Flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from Flask)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from Flask)
  Using cached markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from Flask)
  Using cached werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached click-8.3.3-py3-none-any.whl (110 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, Flask
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.3.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8
```

Figure 15 Flask Installation.

The environment was kept entirely local, with no connection to any external network or live system. The application was intentionally designed to contain the SSTI vulnerability to illustrate how a single insecure coding pattern can lead to server-side code execution and the disclosure of sensitive application configuration data.

Component	Description
Operating System	Kali Linux
Programming Language	Python 3
Web Framework	Flask
Template Engine	Jinja2 (Flask's default built-in engine)
Vulnerable Function	render_ template_string () with unsensitized f-string input
Browser for Testing	Mozilla Firefox
Application Port	http://127.0.0.1:5000
Input Vector	URL query parameter (?name=)

Table 8: SSTI System Configuration

3.3.1 Understanding the Jinja2 Template Engine

Jinja2 is a powerful and widely adopted templating engine for Python web applications. It is Flask's default engine and is responsible for generating dynamic HTML pages by combining static template markup with runtime data. Jinja2 uses a specific syntax to evaluate expressions and execute logic: content placed inside double curly braces `{{ }}` is treated as an expression and evaluated server-side before the HTML is sent to the client's browser (Abhijeet, 2025).

This behavior is by design and is the feature that makes Jinja2 useful. However, it becomes severe security vulnerability when user-supplied input is passed directly into `render_template_string ()` without sanitization. In such cases, an attacker can embed Jinja2 expression syntax into their input, and the server will evaluate it as part of the application's own template code rather than treating it as plain text data.

3.3.2 The Vulnerable Application

The vulnerable application was built as a simple personalized greeting page. It reads a name parameter from the URL query string and uses it to generate a welcome message. The application was deliberately written using an insecure pattern in which the user input is embedded into the template string using Python f-string before being passed to `render_template_string ()`. The critical vulnerable line is:

```
return render_template_string(f"<h1>Welcome, {user_input}!</h1>")
```

The problem with this approach is the order of operations. The Python f-string substitutes the `user_input` value into the raw template string first, before Jinja2 processes it. This means that if a user supplies Jinja2 expression syntax as their name, it becomes part of the template itself. When `render_template_string ()` processes the combined string, it finds what appears to be a legitimate Jinja2 expression and evaluates it on the server.

A correctly written version of this code would pass user input as a separate variable into a pre-defined template, keeping the template structure and the user data completely separate. The insecure pattern used here is, however, a documented real-world mistake that developers unfamiliar with Jinja2's execution model commonly make.

3.3.3 Attack Payloads Used

The SSTI attack was carried out by injecting Jinja2 expression syntax through the `?name=URL` parameter. Three payloads were prepared to demonstrate escalating levels of information exposure:

S.N.	Payload Injected via URL	Intended Outcome
1	<code>?name={{7*7}}</code>	Confirm SSTI: server evaluates <code>7*7</code> and returns 49 instead of the literal string
2	<code>?name={{config}}</code>	Dump Flask's internal configuration dictionary, including secret keys and environment variables
3	<code>?name={{self._dict_}}</code>	Access internal object attributes to demonstrate the depth of server-side object access

Table 9: SSTI Payloads and Their Intent

The first payload, `7*7}}`, is the standard proof-of-concept test for SSTI. If the server returns 49 rather than the literal text `{{7*7}}`, it confirms that the template engine is evaluating the injected expression. This is analogous to the single-quote text used in SQL injection reconnaissance. The second payload escalates the attack by accessing Flask's `config` object, which in a real-world deployment could expose secret cryptographic keys, database connection strings, and other sensitive environment variables. The detailed results of each payload are presented in Section 4.3.

4 Mitigation

SQL Injection received the maximum CVSS score of 10.0 Critical, reflecting that it is remotely exploitable with no authentication, no user interaction, and low complexity, while producing complete impact across all three security pillars, confidentiality, integrity, and availability. The scope change from the web application layer to the database further elevates the score. The mitigation applied, replacing the vulnerable query with prepared statements, directly eliminates the root cause by ensuring user input can never alter the query structure, regardless of its content.

Base Score

10.0 (Critical)

Attack Vector (AV)
Network (N) Adjacent (A) Local (L) Physical (P)

Attack Complexity (AC)
Low (L) High (H)

Privileges Required (PR)
None (N) Low (L) High (H)

User Interaction (UI)
None (N) Required (R)

Scope (S)
Unchanged (U) Changed (C)

Confidentiality (C)
None (N) Low (L) High (H)

Integrity (I)
None (N) Low (L) High (H)

Availability (A)
None (N) Low (L) High (H)

Figure 16: CVSS Score

Metric	Selection	Justification
Attack Vector	Network (N)	Exploited remotely through a web browser, no physical access required
Attack Complexity	Low (L)	Simple payload requiring no special conditions or prior knowledge
Privileges Required	None (N)	Attack targets the login page itself, no account needed
User Interaction	None (N)	Attackers operate alone with no victim involvement
Scope	Changed (C)	An attack crosses from the web application into the backend database
Confidentiality	High (H)	All user credentials and sensitive data can be extracted
Integrity	High (H)	Database records can be modified or deleted
Availability	High (H)	Tables can be dropped, causing full application failure
CVSS Base Score	10.0 – Critical	Maximum severity rating

Table 10: CVSS v3.1 Score

4.1 Test 1: SQL attack Input Sanitization

Input Sanitization is understood as data sanitization, which refers to cleaning the user inputs to ensure safety. It does not affect the input devices, but rather it involves removing, deleting, or modifying the data that could lead to system errors and vulnerabilities, and sanitizing means ensuring that the system remains safe from malicious inputs. (Buenning, 2024)

4.1.1 Step – 1 Temporarily Making it Vulnerable Again for Screenshots

As demonstrated in Section 3.1, the login page was deliberately built with a vulnerable SQL query that directly embedded user-supplied input into the query string without any validation or parameterization. The backend PHP code responsible for this vulnerability was structured as follows:

```
$username = $_POST['username'];
$password = $_POST['password'];

// VULNERABLE QUERY - direct string concatenation
$query = "SELECT * FROM users WHERE username='$username' AND password='$password'";
$result = mysqli_query($conn, $query);
```

Figure 17: VS Code showing the vulnerable query.

Because the variables `$username` and `$password` are taken directly from the HTTP POST request and concatenated into the SQL string, the database has no way of distinguishing between intended query logic and attacker-supplied input. When the payload `' OR '1'='1'` was entered as the username with any password, the database received and executed the following manipulated query:

SELECT * FROM users WHERE= ' ' OR '1'='1' AND password='pass123'

Since condition `1'='1'` is always true, the OR operator caused the entire WHERE clause to evaluate as true for every row in the database. As a result, the application returned to a successful login response without any valid credentials being provided.

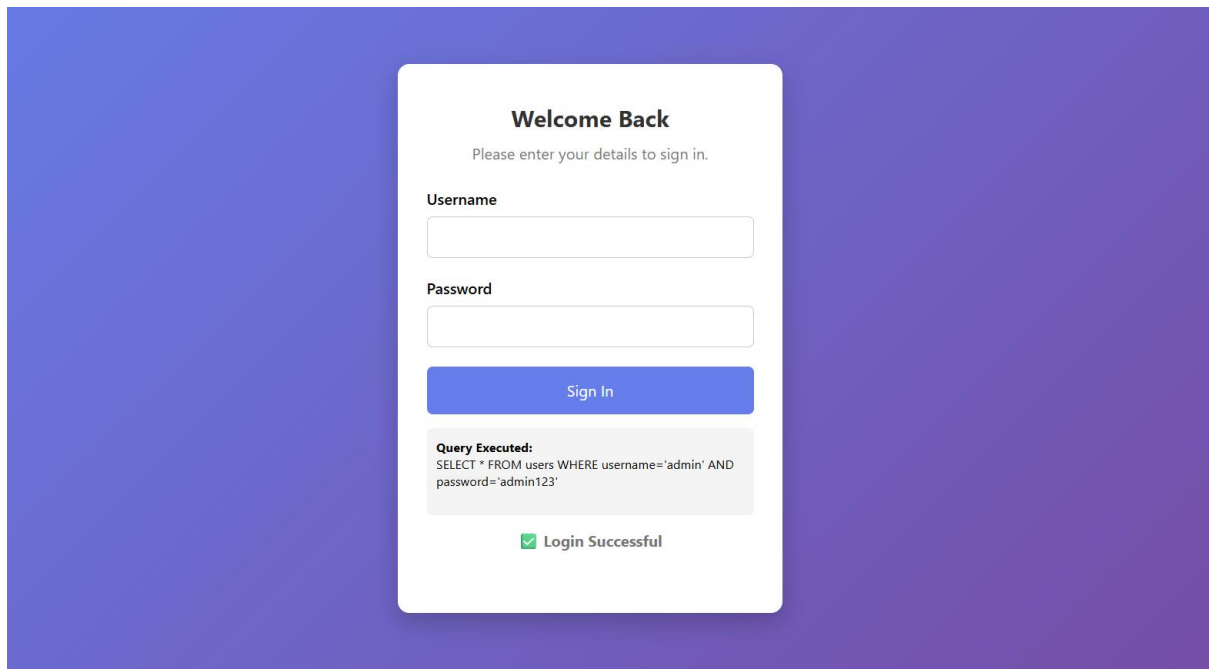


Figure 18: Injection login working with admin/admin123.

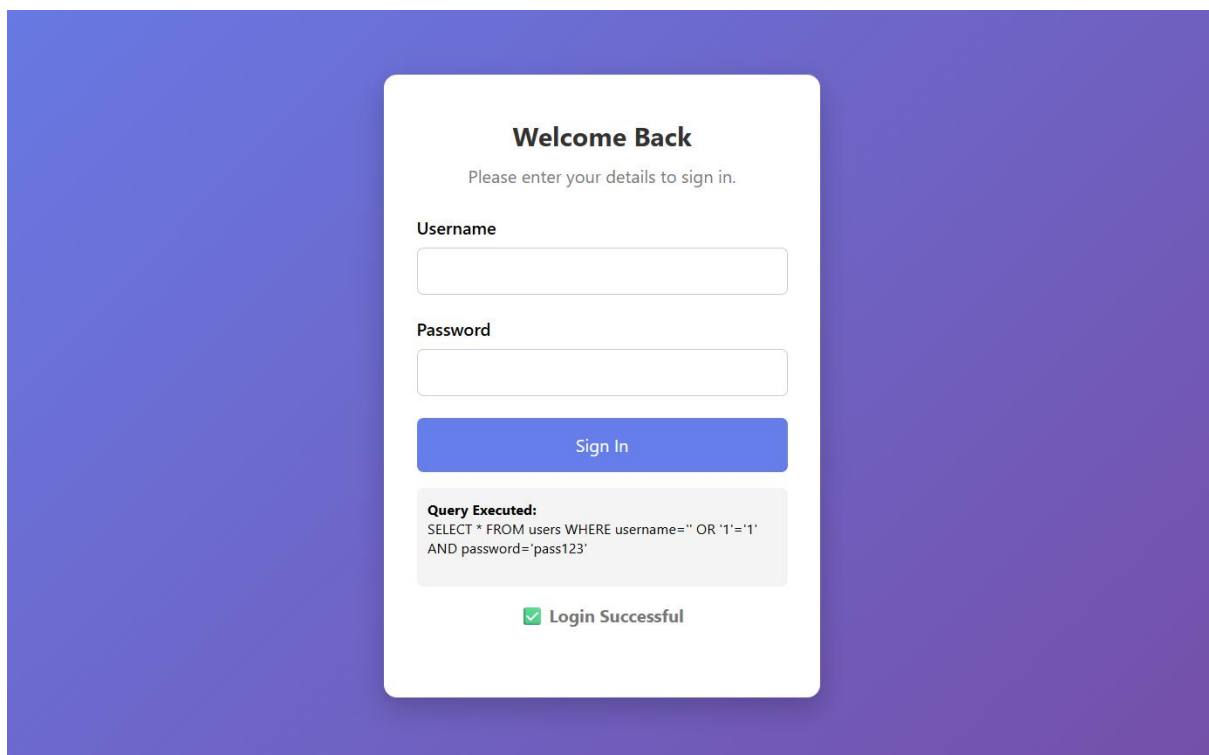


Figure 19: Injection payload 'OR'1'='1

The vulnerability occurred in the login page when the attacker directly inserted a payload ' OR '1'='1 into the user input field, and the login page was vulnerable as it exploited the page, and the login was successful.

As a result, malicious payloads such as ' OR '1'='1 allowed the attacker to exploit the system.

Payload	'OR '1'='1
---------	------------

4.1.2 Step – 2 Prepared Statements with Parameterized Queries

To eliminate this vulnerability, the insecure query was replaced with a prepared statement using PHP's `mysqli_prepare()` function. Prepared statements work by separating the SQL query structure from the user-supplied data entirely. The query is defined first with placeholder characters (?) in place of actual values, and the real input is bound separately afterward. This means the database receives and compiles the query structure before it ever sees the user's input, making it impossible for injected SQL syntax to alter the query logic (Halfond & Orso, 2020).

```

5  $username = $_POST['username'];
6  $password = $_POST['password'];
7
8  // SECURE PREPARED STATEMENT
9  $stmt = mysqli_prepare(
10     $conn,
11     "SELECT * FROM users WHERE username=? AND password=?"
12 );
13
14 mysqli_stmt_bind_param($stmt, "ss", $username, $password);
15
16 mysqli_stmt_execute($stmt);
17
18 $result = mysqli_stmt_get_result($stmt);
19
20 // Display query safely (for coursework/demo)
21 $queryDisplay = "<b>Prepared Statement Used:</b><br>
22                 <code>SELECT * FROM users WHERE username=? AND password=?</code><br><br>";
23

```

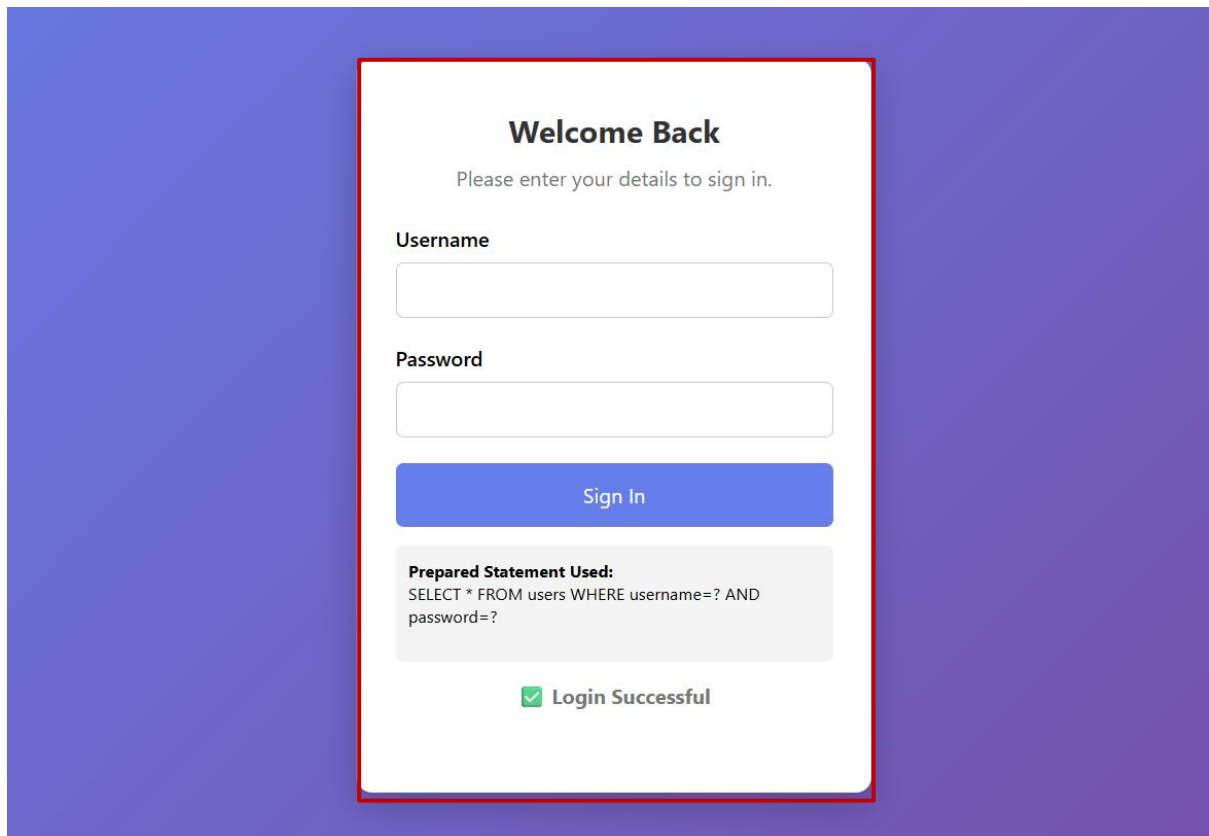
Figure 20: VS Code showing the prepared statement fixed code.

Component	Purpose
<code>Mysqli_prepare()</code>	Sends the query structure to the database with ? placeholders, no user data is included at this stage
<code>Mysqli_stmt_bind_param()</code>	Attaches the actual username and password values to the placeholders as string parameters ("ss")
<code>Mysqli_stmt_execute()</code>	Executes the compiled statement; user input is treated strictly as data, never as SQL logic
<code>Mysqli_stmt_get_result()</code>	Retrieves the result set from the executed statement

Table 11: Break down of each fixed component.

4.1.3 Step – 3 Valid Login (admin/ admin123)

The login was tested first with legitimate credentials to confirm that the fix did not break normal application functionality. The application returned a successful login response as expected, confirming that prepared statements do not interfere with valid user authentication.



The screenshot shows a login interface with a white card on a purple gradient background. The card has a red border. At the top, it says 'Welcome Back' in bold, followed by 'Please enter your details to sign in.' Below this are two input fields: 'Username' and 'Password'. A blue 'Sign In' button is positioned below the password field. Under the button, a grey box displays the 'Prepared Statement Used:' as 'SELECT * FROM users WHERE username=? AND password=?'. At the bottom of the card, a green checkmark icon is next to the text 'Login Successful'.

Figure 21: Normal login is still working after the fix.

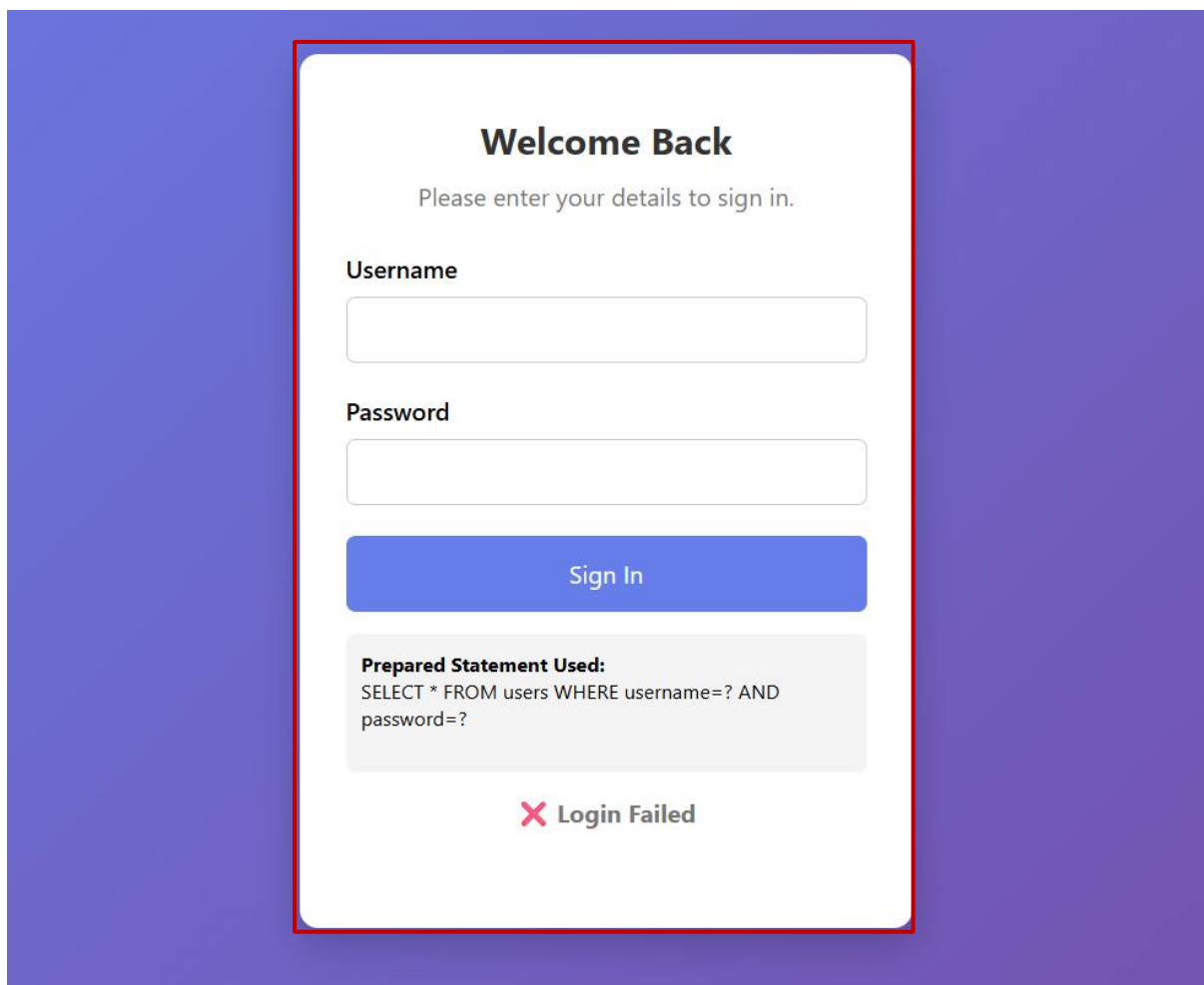
4.1.4 Step – 4 Injection Payload (' OR '1'='1/ pass123)

The same payload that had previously bypassed authentication was submitted again after the fix was applied. Unlike before, the application now returned a Login Failed response. The query display box on the login page confirmed that a prepared statement was used rather than a raw concatenated query, showing:

```
"SELECT * FROM users WHERE username=? AND password=?"
```

Figure 22: Prepared Statement Used

The payload was treated as a plain text string with no effect on the query structure, and no rows were returned from the database.



Welcome Back

Please enter your details to sign in.

Username

Password

Sign In

Prepared Statement Used:
SELECT * FROM users WHERE username=? AND password=?

✗ Login Failed

Figure 23: Injection payload now blocked, showing Login Failed

Test	Input Used	Before Mitigation	After Mitigation
Normal Login	admin/ admin123	Login Successful	Login Successful
SQL Injection	' OR '1'='1/ pass123	Login Successful (bypassed)	Login Failed (blocked)

Table 12: Summary of Test 1

4.2 Test 2: OS Command Injection

OS command injection is a critical web application vulnerability, which allows attackers to execute arbitrary Operating system commands on the server running an application (portswigger, 2026).

4.2.1 Step 1: Creation and Activation of Virtual Environment

Here for this test, a Python Virtual Environment needs to be set up, and to avoid any conflict with Kali's system packages, a new virtual environment is created and activated.

The Virtual environment is named 'venv'.

Python3 -m venv venv

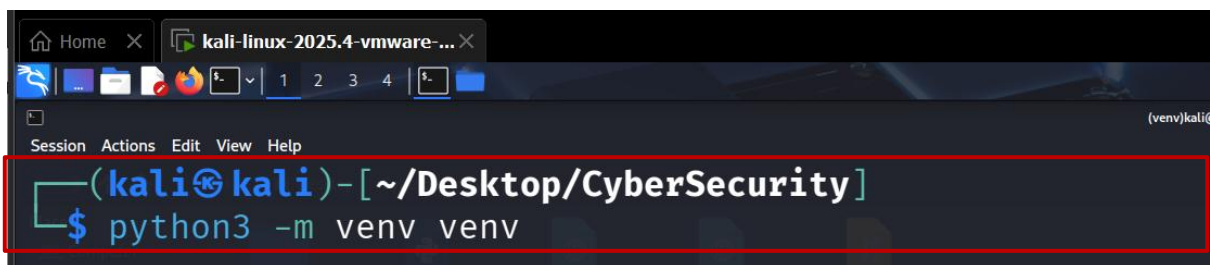
A terminal window titled 'kali-linux-2025.4-vmware-...' is shown. The prompt is '(kali@kali)-[~/Desktop/CyberSecurity]'. The command 'python3 -m venv venv' has been entered and is highlighted with a red box. The terminal also shows a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'.

Figure 24 Virtual Environment creation.

Now to activate the virtual Environment 'venv', source venv/bin/activate is typed in terminal.

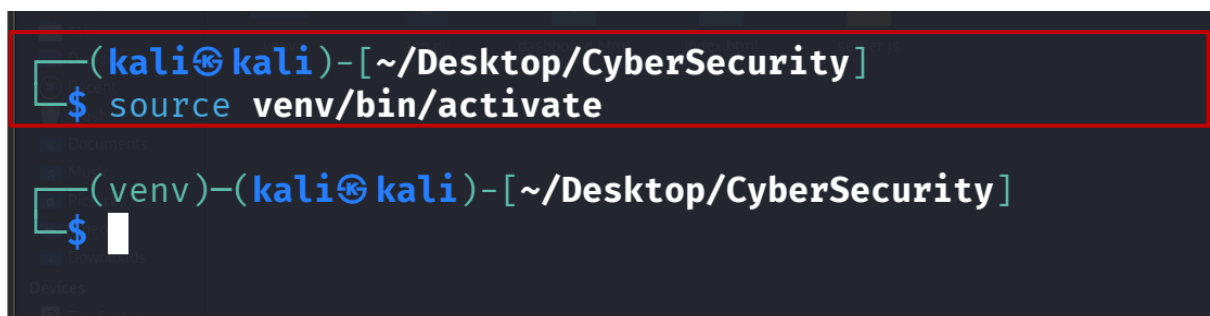
A terminal window showing the activation of the virtual environment. The prompt is '(kali@kali)-[~/Desktop/CyberSecurity]'. The command 'source venv/bin/activate' has been entered and is highlighted with a red box. Below this, the prompt has changed to '(venv)-(kali@kali)-[~/Desktop/CyberSecurity]', indicating the virtual environment is now active.

Figure 25 Activating Virtual Environment

4.2.2 Step 2: Installation of Flask in the Virtual Environment

The command used to install flask is “**pip install Flask**” in terminal.

```
(venv)-(kali@kali)-[~/Desktop/CyberSecurity]
$ pip install Flask
Collecting Flask
  Using cached flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from Flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from Flask)
  Using cached click-8.3.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from Flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from Flask)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from Flask)
  Using cached markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from Flask)
  Using cached werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached click-8.3.3-py3-none-any.whl (110 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, Flask
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.3.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8
```

Figure 26 Flask Installation

4.2.3 Step 3: Creating the Vulnerable Python Application using nano command.

Here, a vulnerable Python Application is made.

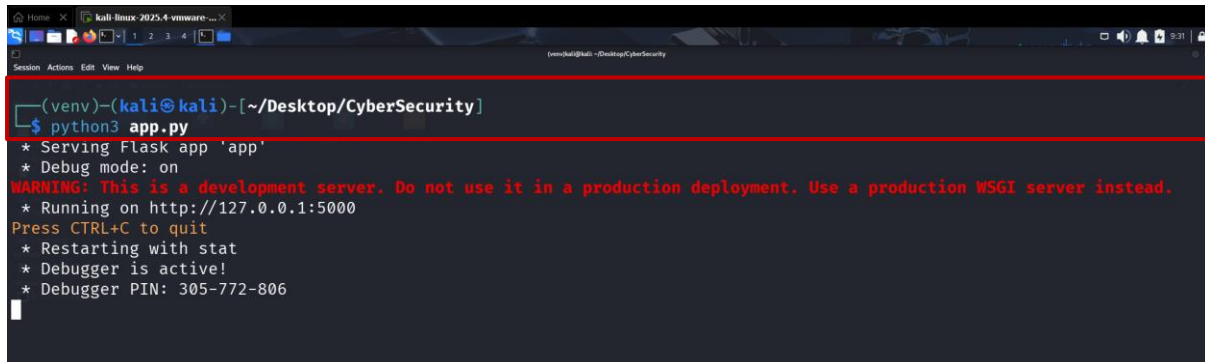
```
1 from flask import Flask, request
2 import subprocess
3
4 app = Flask(__name__)
5
6 HTML_FORM = """
7 <h2>Network Diagnostic Tool</h2>
8 <p>Enter an IP address to ping:</p>
9 <form method="POST" action="/ping">
10   <input type="text" name="ip_address" placeholder="e.g., 127.0.0.1">
11   <button type="submit">Ping</button>
12 </form>
13 """
14
15 @app.route('/')
16 def index():
17     return HTML_FORM
18
19 @app.route('/ping', methods=['POST'])
20 def ping_server():
21     user_input = request.form.get('ip_address')
22
23     # THE VULNERABILITY: Blindly trusting input in a shell command
24     command = "ping -c 1 " + user_input
25
26     try:
27         output = subprocess.check_output(command, shell=True, text=True, stderr=subprocess.STDOUT)
28         return f"<h3>Results:</h3><pre>{output}</pre>"
29     except subprocess.CalledProcessError as e:
30         return f"<h3>Error:</h3><pre>{e.output}</pre>"
31
32 if __name__ == '__main__':
33     app.run(debug=True, port=5000)
34
```

Figure 27 Creation of Vulnerable Python Application using nano command.

The code used in creating the vulnerable python application is in [Appendix](#)

4.2.4 Step 4: Running the Vulnerable Server

The command used to run the vulnerable Server `python3 app.py`

A terminal window titled 'kali-linux-2025.4-vmware-...' shows the command 'python3 app.py' being executed. The output indicates that a Flask app named 'app' is being served in debug mode on http://127.0.0.1:5000. A warning message states: 'WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.' The terminal also shows that the server is restarting with stat, the debugger is active, and the debugger PIN is 305-772-806.

```
(venv)-(kali@kali)-[~/Desktop/CyberSecurity]
$ python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 305-772-806
```

Figure 28 Running the Vulnerable Server

4.2.5 Step 5: Running on the Firefox

After the vulnerable server, <http://127.0.0.1:5000> is pasted in Firefox and a page is opened.

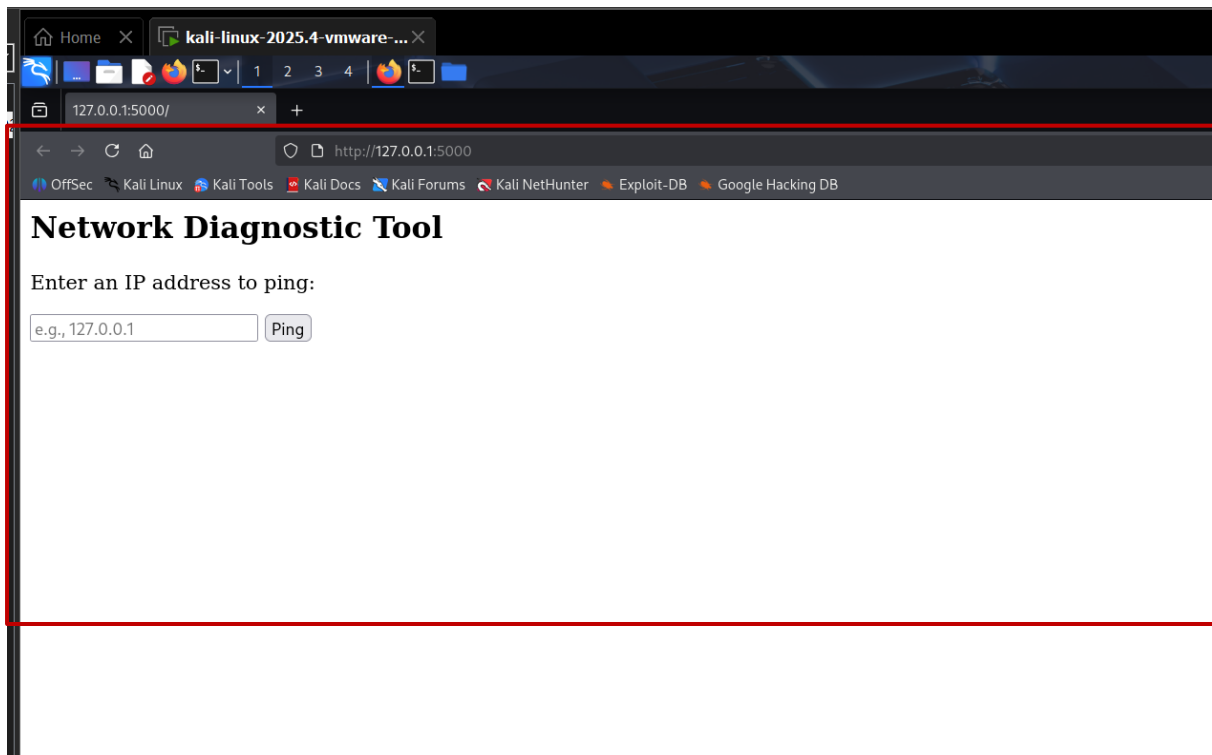


Figure 29 Firefox opened.

4.2.6 Step 6: Checking Ping

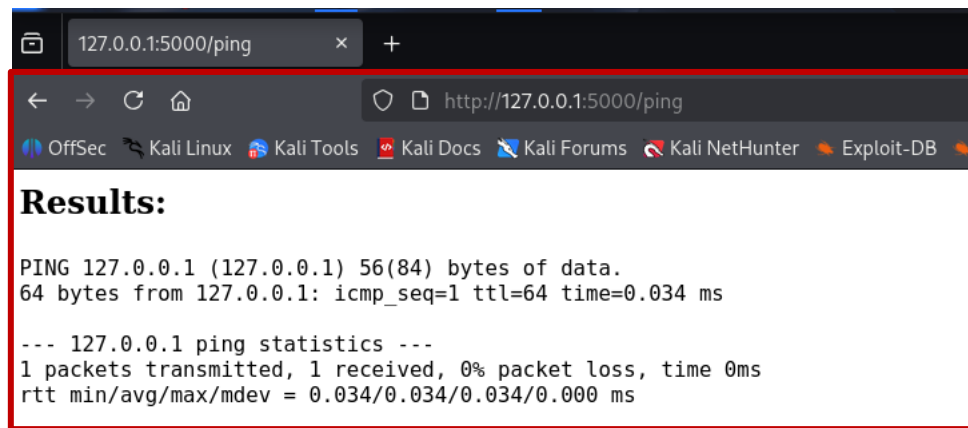


Figure 30 Checking Ping

4.2.7 Step 7: Trying Payload 1

Running the link **127.0.0.1; cat /etc/passwd** on the text field

As it shows the contents of the Kali system's password file on the web page

```

127.0.0.1:5000/ping
http://127.0.0.1:5000/ping
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Results:

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.055 ms

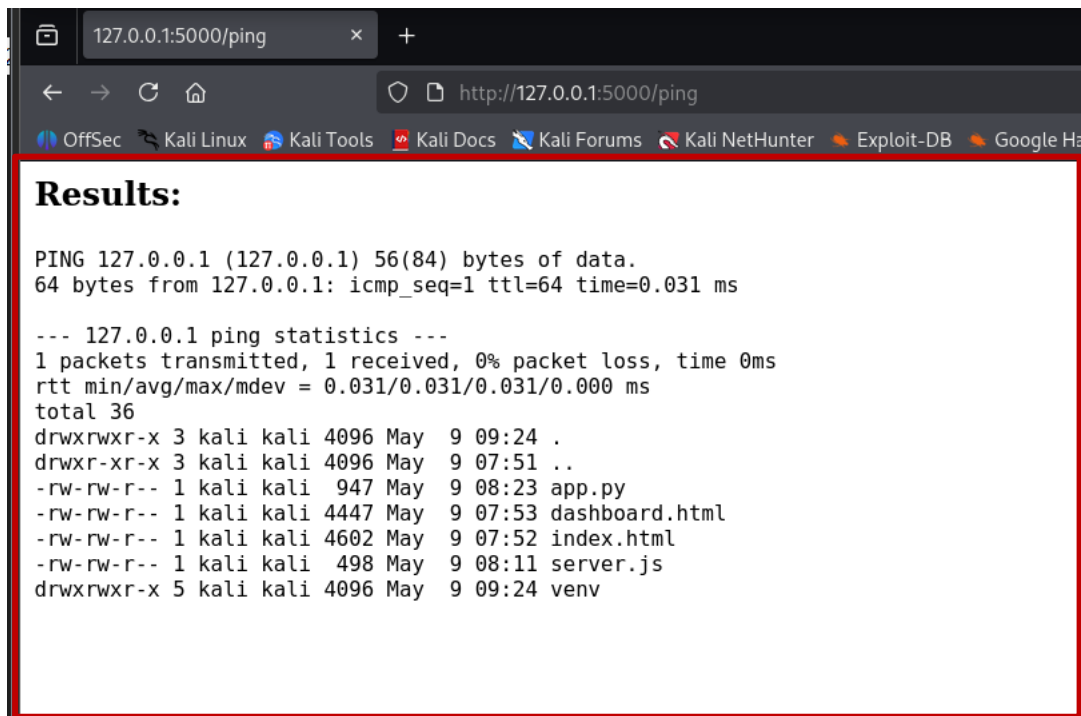
--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.055/0.055/0.055/0.000 ms
/bin/sh: 1: whoami: not found
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534:./nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:./usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon:/usr/lib/dhcpcd:/bin/false
systemd-timesync:x:991:991:systemd Time Synchronization:./usr/sbin/nologin
messagebus:x:990:990:System Message Bus:/nonexistent:/usr/sbin/nologin
tss:x:101:104:TPM software stack:/var/lib/tpm:/bin/false
strongswan:x:102:65534:./var/lib/strongswan:/usr/sbin/nologin
tcpdump:x:103:105:./nonexistent:/usr/sbin/nologin
sshd:x:989:65534:sshd user:/run/ssh:/usr/sbin/nologin
_rpc:x:104:65534:./run/rpcbind:/usr/sbin/nologin
dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin
avahi:x:105:108:Avahi mDNS daemon:/run/avahi-daemon:/usr/sbin/nologin
nm-openvpn:x:106:109:NetworkManager OpenVPN:/var/lib/openvpn/chroot:/usr/sbin/nologin
speech-dispatcher:x:107:29:Speech Dispatcher:/run/speech-dispatcher:/bin/false
usbmux:x:108:46:usbmux daemon:/var/lib/usbmux:/usr/sbin/nologin
nm-openconnect:x:988:988:NetworkManager OpenConnect plugin:./usr/sbin/nologin
pipewire:x:987:987:system user for pipewire:/nonexistent:/usr/sbin/nologin
lightdm:x:109:110:Light Display Manager:/var/lib/lightdm:/bin/false
statd:x:110:65534:./var/lib/nfs:/usr/sbin/nologin
saned:x:111:111:./var/lib/saned:/usr/sbin/nologin
polkitd:x:986:986:User for polkitd:./usr/sbin/nologin
rtkit:x:112:112:RealtimeKit:/proc:/usr/sbin/nologin
colord:x:985:985:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin
pcscd:x:984:984:PC/SC Smart Card Daemon:./usr/sbin/nologin
mysql:x:113:114:MariaDB Server:/nonexistent:/bin/false
stunnel4:x:983:983:stunnel service system account:/var/run/stunnel4:/usr/sbin/nologin
geoclue:x:114:115:./var/lib/geoclue:/usr/sbin/nologin
Debian-snmpp:x:115:116:./var/lib/snmpp:/bin/false
ssldh:x:116:117:./nonexistent:/usr/sbin/nologin
cups-pk-helper:x:117:120:user for cups-pk-helper service:/nonexistent:/usr/sbin/nologin
redsocks:x:118:121:./var/run/redsocks:/usr/sbin/nologin
_gophish:x:119:123:./var/lib/gophish:/usr/sbin/nologin
iodine:x:120:65534:./run/iodine:/usr/sbin/nologin
miredo:x:121:65534:./var/run/miredo:/usr/sbin/nologin
redis:x:122:124:./var/lib/redis:/usr/sbin/nologin

```

Figure 31 Result of Payload 1

4.2.8 Step 8: Trying Payload 2

Running the link 127.0.0.1; ls -la which will list all files in the directory where the app.py is currently running.



```

127.0.0.1:5000/ping
http://127.0.0.1:5000/ping
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Ha

Results:

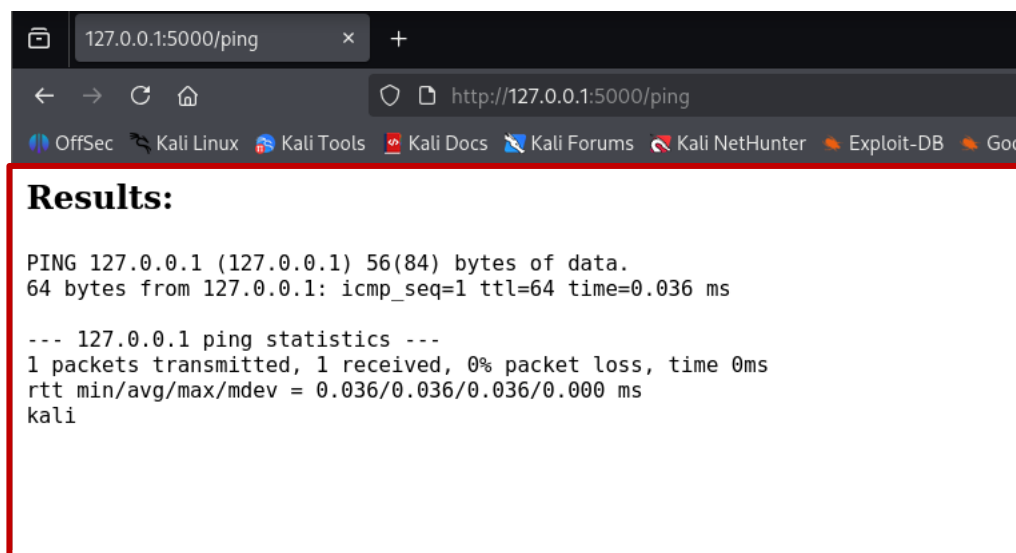
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.031 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.031/0.031/0.031/0.000 ms
total 36
drwxrwxr-x 3 kali kali 4096 May  9 09:24 .
drwxr-xr-x 3 kali kali 4096 May  9 07:51 ..
-rw-rw-r-- 1 kali kali  947 May  9 08:23 app.py
-rw-rw-r-- 1 kali kali 4447 May  9 07:53 dashboard.html
-rw-rw-r-- 1 kali kali 4602 May  9 07:52 index.html
-rw-rw-r-- 1 kali kali  498 May  9 08:11 server.js
drwxrwxr-x 5 kali kali 4096 May  9 09:24 venv
  
```

Figure 32 Result of Payload 2

4.2.9 Step 9: Trying Payload 3

Running the command 127.0.0.1 && whoami which will show the root or username.



```

127.0.0.1:5000/ping
http://127.0.0.1:5000/ping
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB God

Results:

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.036 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.036/0.036/0.036/0.000 ms
kali
  
```

Figure 33 Result of Payload 3

4.2.10 Step 10: Mitigation Method (Removing Shell=True and Parameterizing the Command.)

The cause of the vulnerability is in the code, where there was `shell=True` which was combined with string concatenating (`"ping -c 1 " + user_input`). This command tells python to pass the raw string to `/bin/sh`, which interprets the injection characters.

To fix this by passing the command and its arguments as a list (an array), rather than a single string and explicitly set the `shell=False`.

When the list is passed to `sub process`, python bypasses the shell entirely. It directly executes the target binary (ping) and feeds it the exact arguments in the list. Special characters like `;` or `&&` lose their magic because they are treated strictly as literal text strings, not command operators.

4.2.11 Step 11: Implementation of Strict Input Validation which allows listing.

Even if the parameterized command is there, user should never blindly trust user input. As if the application expects an IP address, user must violently reject anything which is not valid (IP address), before the input gets anywhere near the `subprocess` module.

To fix this, Use of Python's built-in `ip address` module to cryptographically verifying the input format can help. If the input contains letters, space or semicolons, the module will immediately throw an error, which will stop the attack.

4.2.12 Step 12: Applying the Patched Code

The patched code is [here](#)

Now Testing the Mitigation Process

Pinging first

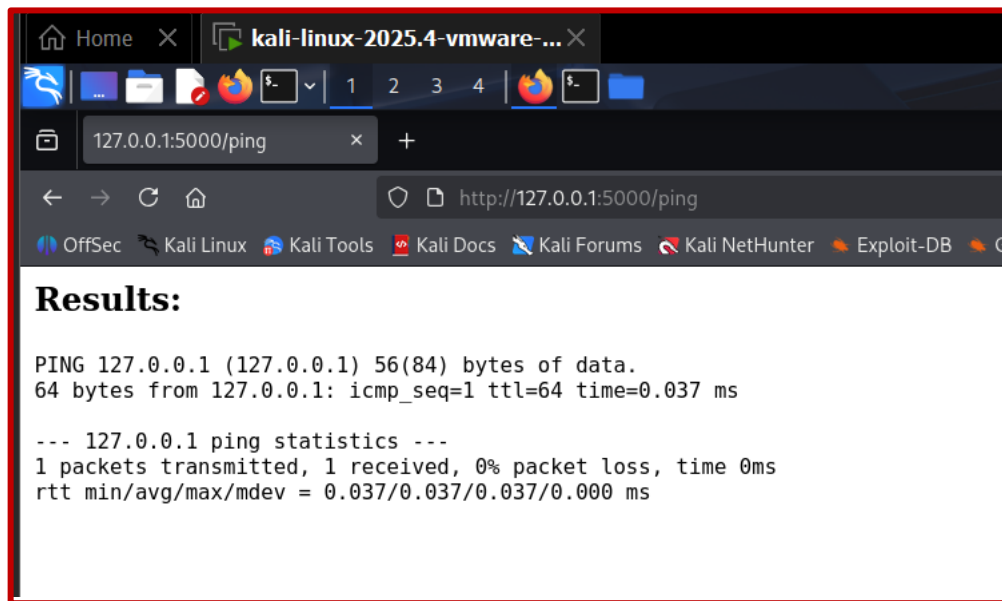


Figure 34 Ping.

It works the same, now evaluating the Payloads.

4.2.13 Step 13: Testing Payload 1

Testing payload 1 127.0.0.1; cat /etc/passwd on the patched one

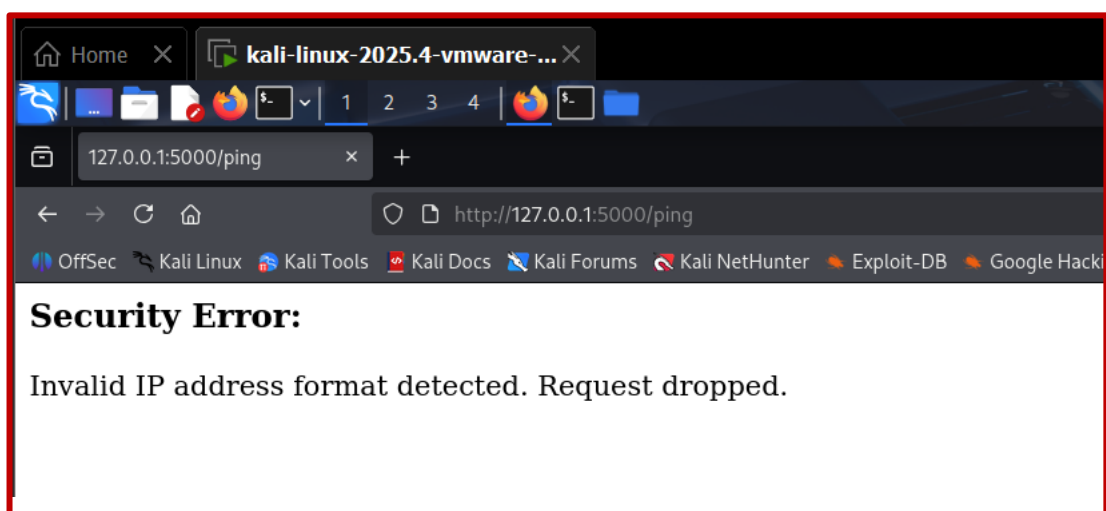


Figure 35 Testing Payload 1 after applying mitigation method.

4.2.14 Step 14: Testing Payload 2

Testing payload 2: 127.0.0.1; ls -la

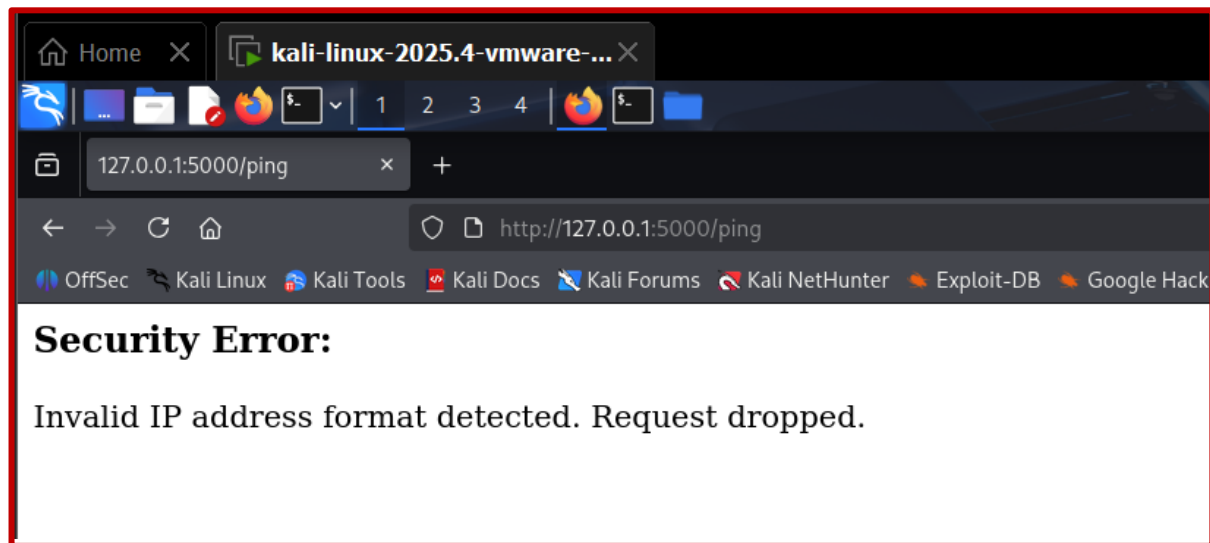


Figure 36 Testing Payload 2 after applying mitigation method.

4.2.15 Step 15: Testing Payload 3

Testing Payload 3 127.0.0.1 && whoami

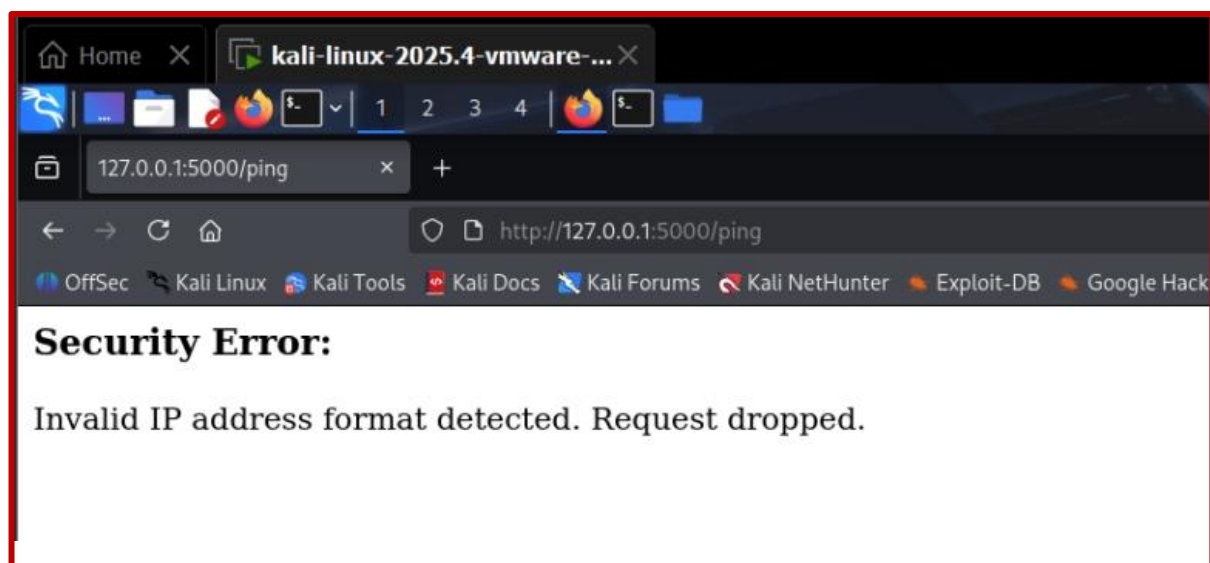


Figure 37 Testing Payload 3 after applying mitigation method.

4.3 Test 3: Server-Side Template Injection

4.3.1 Step 1: Installing Flask

To test Server-Side Template, Flask is installed, in the first step.

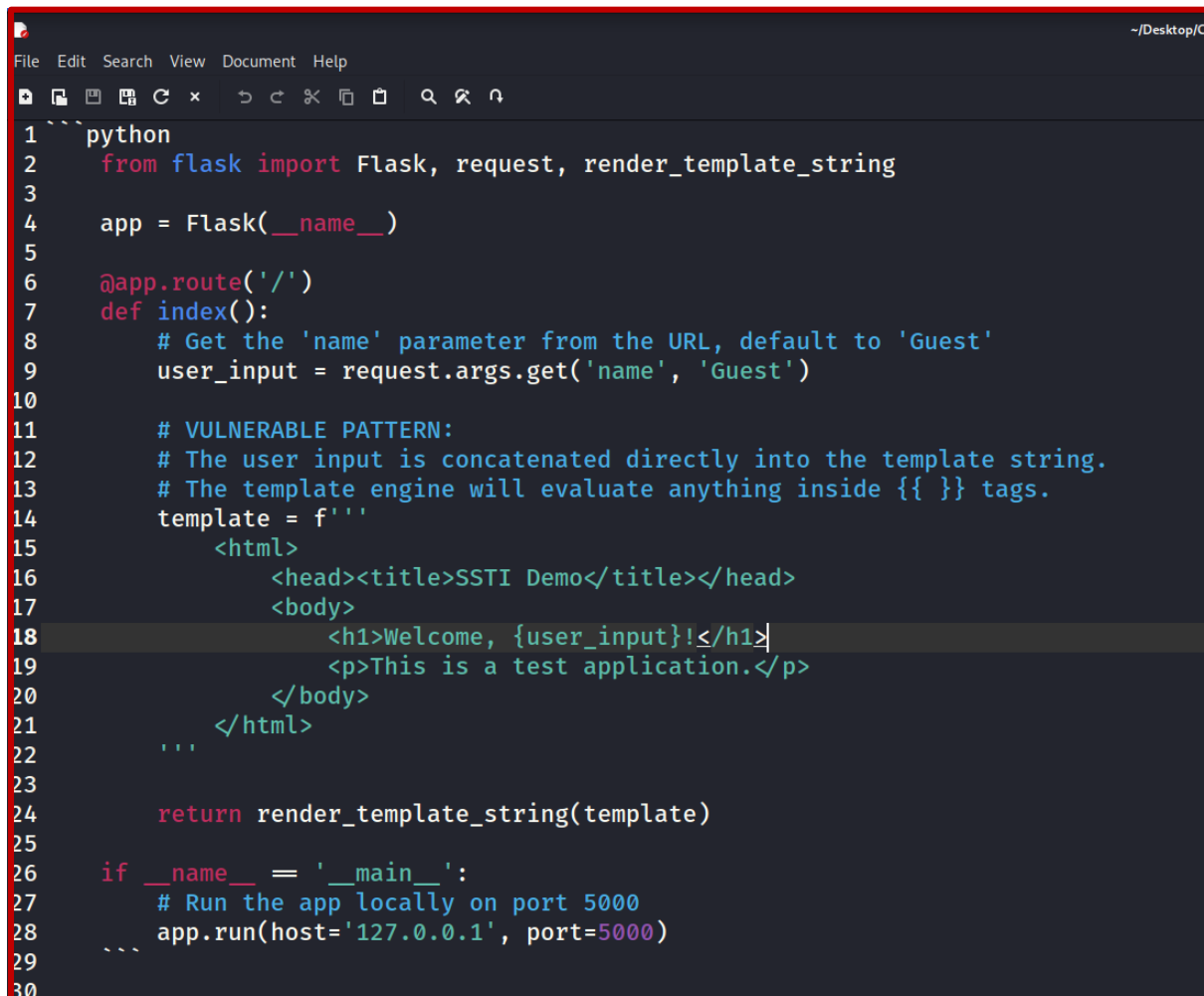


```
(venv)-(kali@kali)-[~/Desktop/CyberSecurity]
$ pip install Flask
Collecting Flask
  Using cached flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from Flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from Flask)
  Using cached click-8.3.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from Flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from Flask)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from Flask)
  Using cached markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from Flask)
  Using cached werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached click-8.3.3-py3-none-any.whl (110 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, Flask
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.3.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8
```

Figure 38 Installing Flask

4.3.2 Step 2: Vulnerable python code.

The next step is to make vulnerable code using python language.



```

1  python
2  from flask import Flask, request, render_template_string
3
4  app = Flask(__name__)
5
6  @app.route('/')
7  def index():
8      # Get the 'name' parameter from the URL, default to 'Guest'
9      user_input = request.args.get('name', 'Guest')
10
11     # VULNERABLE PATTERN:
12     # The user input is concatenated directly into the template string.
13     # The template engine will evaluate anything inside {{ }} tags.
14     template = f'''
15         <html>
16             <head><title>SSTI Demo</title></head>
17             <body>
18                 <h1>Welcome, {user_input}!</h1>
19                 <p>This is a test application.</p>
20             </body>
21         </html>
22     '''
23
24     return render_template_string(template)
25
26 if __name__ == '__main__':
27     # Run the app locally on port 5000
28     app.run(host='127.0.0.1', port=5000)
29
30

```

Figure 39 Vulnerable Python Code

4.3.3 Step 3: Run the python3 application and open the site.

After making vulnerable code, let's run application python3 app.py and open the site.



```

kali@kali: ~/Desktop/CyberSecurity
$ python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
27.0.0.1 - - [14/May/2026 00:52:22] "GET / HTTP/1.1" 200 -

```

Figure 40 Running python3.

4.3.4 Step 4: Open Firefox

The link used is <http://127.0.0.1:5000>

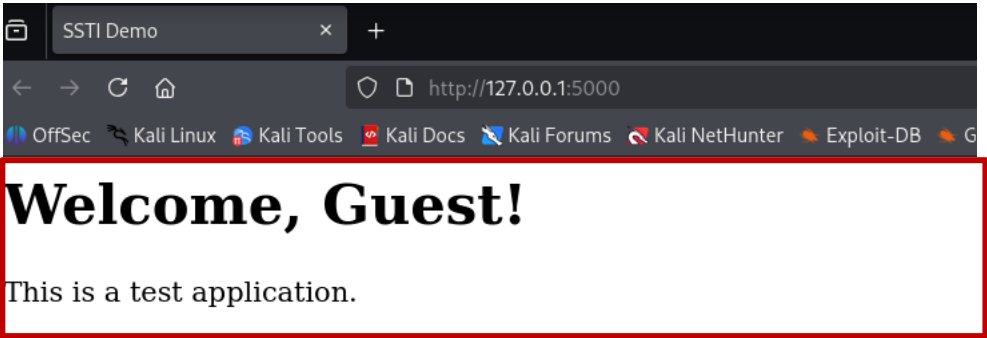


Figure 41 Opening Link in Firefox

4.3.5 Step 5: Changing the URL and Seeing the Result

Before URL	http://127.0.0.1:5000
After URL	http://127.0.0.1:5000/?name=poonam

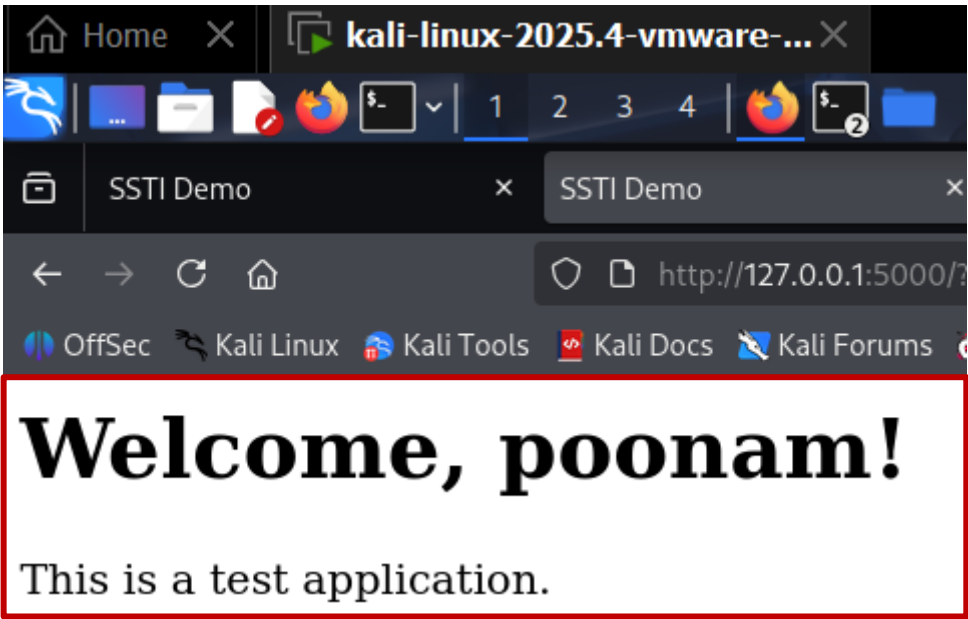


Figure 42 Viewing Result after Changing Result (1)

Result description	The page shows exactly as expected, displaying: poonam after making changes in URL with name=poonam
--------------------	---

Now, to test for Server-side template injection (SSTI), Specific syntax is injected into the template engine. Here Jinja2 uses `{{ ... }}` to evaluate the expression. And here simple mathematical equations are injected, and if the application is vulnerable, the server will evaluate the math before returning the HTML as a result.

Here to test that, `{{ 7*7 }}` is provided as the name of parameter.

Before URL	http://127.0.0.1:5000
After URL	<code>http://127.0.0.1:5000/?name={{7*7}}</code>

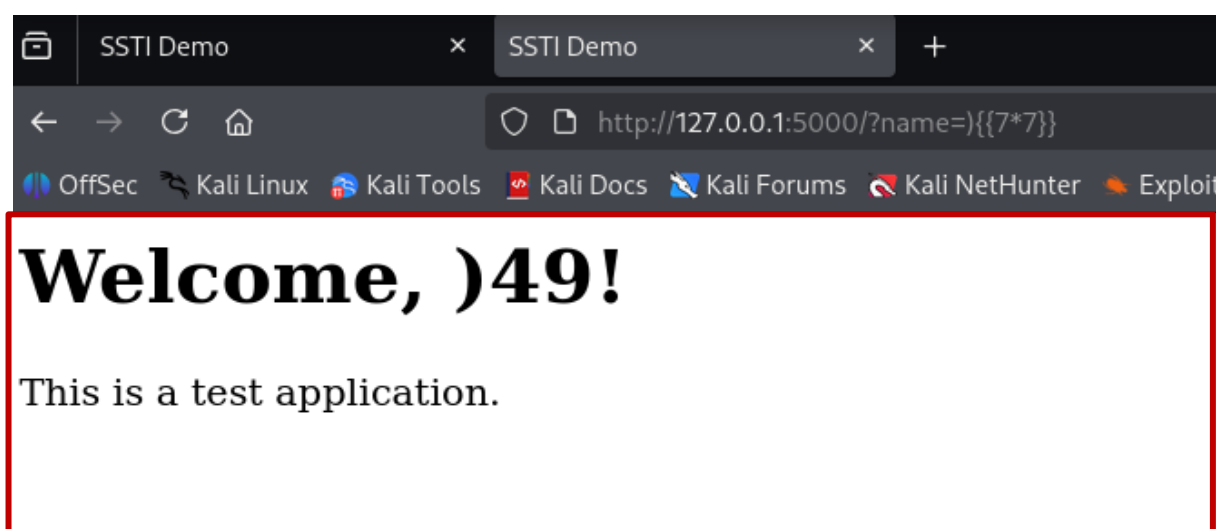


Figure 43 Viewing Result after Changing Result (2)

Result description	The page shows exactly as expected, calculating <code>7*7</code> and showing 49, after making changes in URL with <code>name={{7*7}}</code> , as the server returns <code>Welcome,)49!</code>
--------------------	--

This happened because the python script uses an f-string (`f"Welcome,{user_input}!"`) to build the template.

```

15         <html>
16             <head><title>SSTI Demo</title></head>
17             <body>
18                 <h1>Welcome, {user_input}</h1>
19                 <p>This is a test application.</p>
20             </body>
21         </html>
22     ...

```

Figure 44 f-string in Code.

And the string `{{7*7}}` was placed directly into the raw HTML structure.

When `render_template_string()` processed that final block of code.

```

3
4     return render_template_string(template)
5

```

Figure 45 Final Block of Code

It saw the `{{ }}` tags, and recognized it as jinja2 instructions, and executed the python arithmetic inside it before sending it as a final response to the user.

The information disclosed here is beyond just a simple math $7*7=49$, as an attacker can use this execution context to dump configuration files or can also access sensitive or confidential objects within the application's environment.

For example, In Flask the config object is accessible within the template. As by again injecting the config object by the URL: <http://127.0.0.1:5000/?name={{config}}>

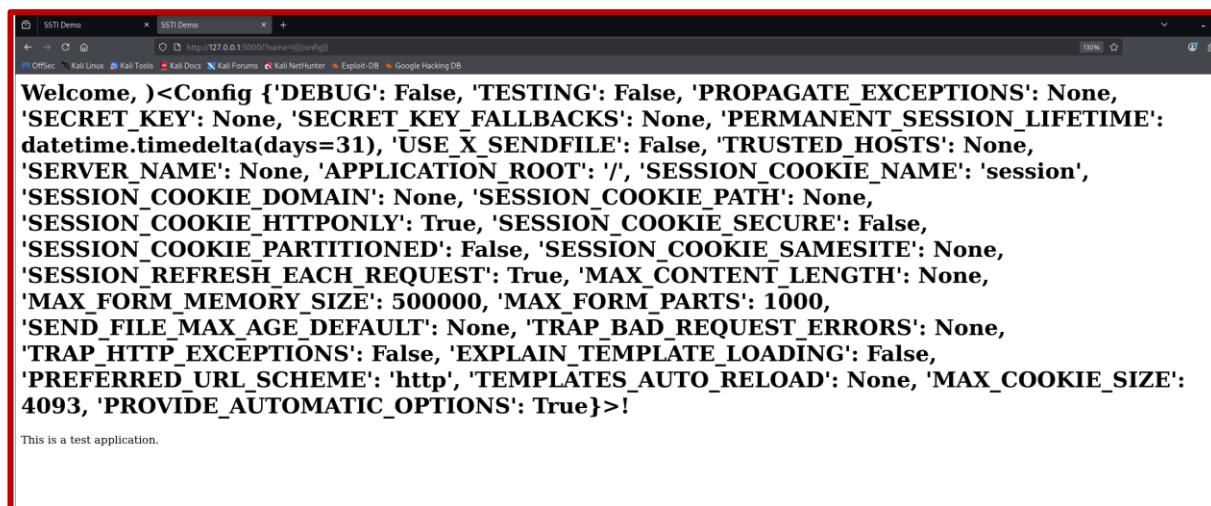
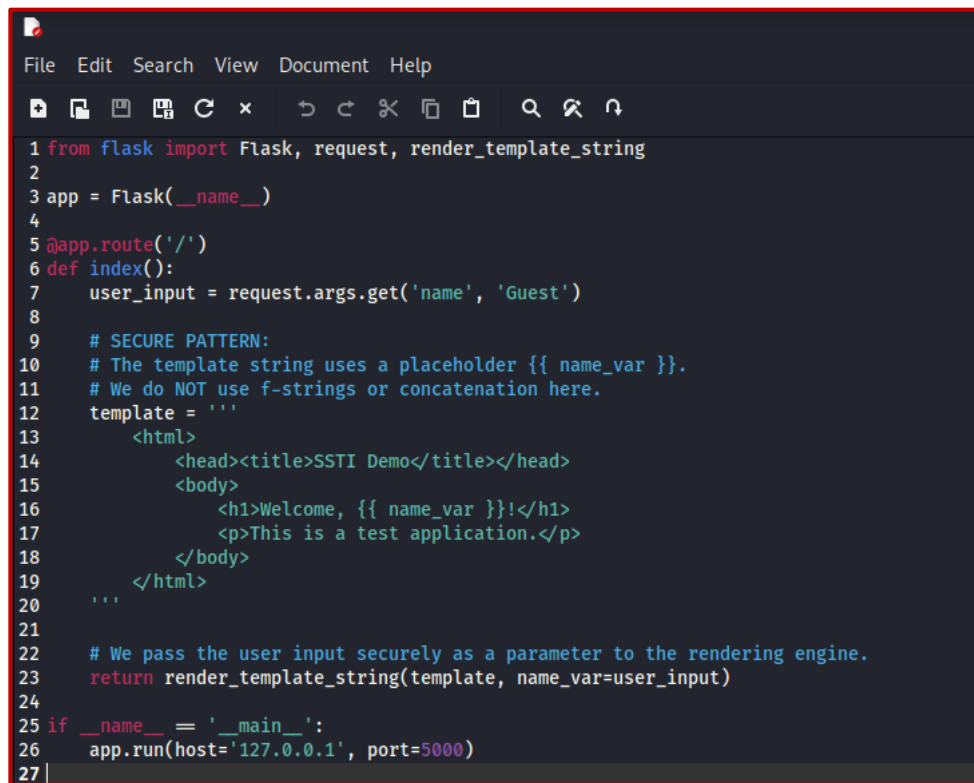


Figure 46 Config Objects

Result description	The application will return the server's internal Flask configuration dictionary as the application was hosted using Flask. In real world scenario, this could expose secret keys, database URL's or other sensitive application environmental variables.
--------------------	---

4.3.6 Step 6: Mitigation process

To mitigate the issue, there is required to have better security code,



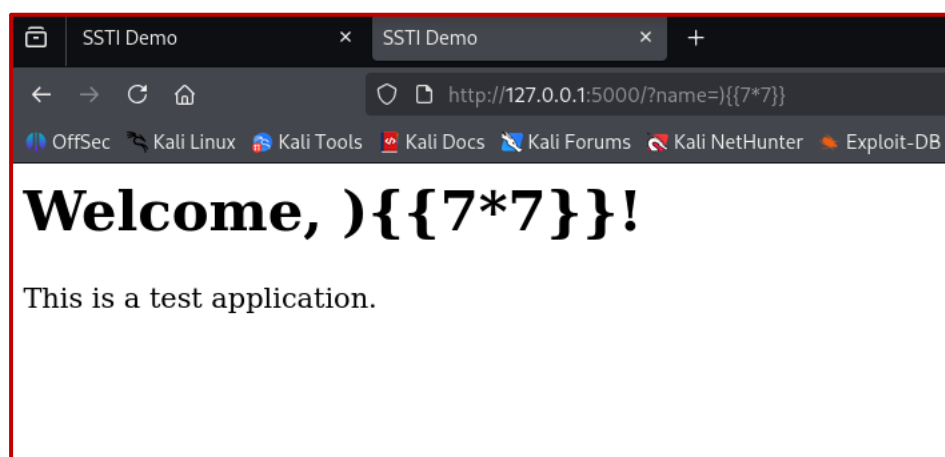
```

1 from flask import Flask, request, render_template_string
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def index():
7     user_input = request.args.get('name', 'Guest')
8
9     # SECURE PATTERN:
10    # The template string uses a placeholder {{ name_var }}.
11    # We do NOT use f-strings or concatenation here.
12    template = '''
13        <html>
14            <head><title>SSTI Demo</title></head>
15            <body>
16                <h1>Welcome, {{ name_var }}!</h1>
17                <p>This is a test application.</p>
18            </body>
19        </html>
20    '''
21
22    # We pass the user input securely as a parameter to the rendering engine.
23    return render_template_string(template, name_var=user_input)
24
25 if __name__ == '__main__':
26     app.run(host='127.0.0.1', port=5000)
27

```

Figure 47 Secure code.

Now returning to the server and checking for the flaws.

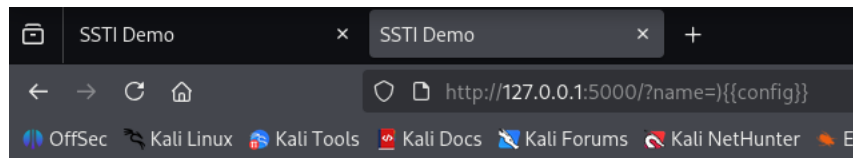


The browser window shows the URL `http://127.0.0.1:5000/?name=){{7*7}}`. The page content is:

Welcome,){{{7*7}}}

This is a test application.

Figure 48 Testing the Secure Code (1)



Welcome,){{{config}}}

This is a test application.

Figure 49 Testing the Secure Code (2)

Result Description	The application now safely treats the input as literal data. It will output <code><h1>Welcome, 7*7}}!</h1></code> . Because the template engine knows <code>name_var</code> is purely data, it strips it of any execution context and refuses to evaluate the math.
--------------------	---

5 Evaluations

Below are the evaluation of all the mitigation steps and techniques we used along with their advantages, disadvantages, and Application Areas.

5.1 Evaluation of Input Sanitization

Sanitization of Input (or Input Filtering/Validation) is an approach to inspect the user's input and eliminate or escape harmful characters before the input is executed in the server-side database.

5.1.1 Advantages of Input Sanitization

- **Defense-in-Depth Approach:** Sanitization acts as a supplementary measure when combined with parameterized queries. Should a programmer fail to implement a prepared statement at one endpoint alone, a sound sanitization technique may come in handy.
- **Ensures Data Consistency (Whitelist):** By strictly validating the input, you can guarantee that the data submitted to the database adheres to the format intended. For instance, if the program needs an age as an input value, you can sanitize the input to be sure it consists only of numbers.
- **Reduces the Need for Parameterization in Legacy Programs:** With legacy programs whose source codes cannot be immediately rewritten with parameterized queries, the only possible solution would be to sanitize inputs centrally.
- **Mitigates "Script Kiddie" Attacks:** Basic sanitization quickly neutralizes low-effort, automated attacks (like those using standard ' OR 1=1 -- payloads) often seen in broad vulnerability scans.

5.1.2 Disadvantages Input Sanitization

- **Bypass High Probability (Blacklist Flaw):** Most sanitization works via "blacklisting," which means that any potentially malicious character, such as ', ", ;, --, is filtered out or blocked. Malicious users usually work around blacklist protection by utilizing URL encoding, Hex encoding, Unicode variations, or SQL dialects' peculiarities.
- **Inconvenience for Legal Input:** Too many aggressive filtering methods distort legal data entered by users. In fact, the famous O'Connor dilemma occurs when an application filters out single quotes; therefore, users whose names contain apostrophes cannot register with an app.
- **Regular Maintenance Required:** Every time hackers develop a new method of penetrating applications' security or database features are updated, sanitization filtering rules need to be altered correspondingly. Designing and maintaining RegExes is a challenging task because one may end up filtering out legitimate input.
- **Dependent on Context:** An apparently innocent payload used in the WHERE clause of a SELECT statement can turn into an extremely malicious payload in case the same input gets incorporated into an ORDER BY statement or a stored procedure.

5.1.3 Application Areas

- **Web Application Firewalls (WAFs):** Web Application Firewalls utilize sanitization and pattern matching techniques extensively to reject any malicious HTTP request prior to hitting the web server or application code.
- **API Gateways/Edge Nodes:** APIs which receive large volumes of untrusted input data from different clients usually perform very rigorous sanitization (allowlisting) on the data to ensure that only sanitized data (with correct types) such as JSON and XML is allowed to pass through into the internal network.
- **Remediating Legacy Code:** In cases where there is legacy code within enterprises that cannot be immediately refactored to follow newer standards, security personnel implement input sanitization wrappers on top of vulnerable database functions.
- **Client-side Sanitization (User Experience Considerations):** Although client-side sanitization (JavaScript executed in the browser) offers no security whatsoever against a hacker (who may easily bypass using Burp Suite or other tools), it is used extensively for providing immediate user feedback if the user enters erroneous data.

5.2 Evaluation of OS Command Injection Mitigation

The evaluation of all the mitigation steps we took is below for OS Command Injections are as follows:

5.3 Command Parameterization.

This method is where a command and arguments are sent one by one in a list instead of a string, and the OS is responsible for executing the target binary's executable without calling the shell interpreter.

5.4 Advantages of Command Parameterization (Avoiding shell=True)

Root Cause Eradication: It is the best protection as it rids vulnerability at the execution layer. It doesn't matter to see a semicolon, an and or an or as any other symbol.

- Language Agnostic: Most of the popular programming language including Python, Java, Node.js and C#, has native libraries for safely running system commands with parameterization.

5.5 Disadvantages of Command Parameterization (Avoiding shell=True)

- Loss of shell features: Using web applications that only run using shell features like output redirection > or piping | between multiple system binaries, parameterization breaks this functionality thus making us must develop code that is more complex to handle data stream manually in application layer.

5.5.1 Application Areas:

APIs and Middleware: Command Parameterization is highly good when building APIs that acts as a bridge between the backend and frontend system utilities.

Hardware and Diagnostic Interfaces: this technique is good for systems that need to trigger specific and diagnostic scripts such as network mapping tools or hardware that is based on user trigger for status check.

5.6 Strict Input Validation (Allow-listing / Type Checking)

The technique is to examine the input carefully, using a set of rules (such as Ip address module that we had used. you can use whatever rules you like), and to aggressively drop anything that fails to match the rules exactly.

5.7 Advantages:

- **Early Termination:** It prevents malicious traffic from entering the application's entrance point, thus conserving system resources, and stopping the malicious payload from penetrating deeper into the system.
- **Clear Error Handling:** It helps the application to fail gracefully and provide useful information back to the user ("Invalid Format," for example).

5.8 Disadvantages:

- **Maintenance Overhead:** If the format the inputs are in is expected to shift, then validation logic must be updated.
- **Allow extreme difficulty for Free-Form Text / complex characters (apostrophes):** If the application needs to support free-form text / complex characters (apostrophes), it will be very difficult to implement the allowlist.
- **Bad validation logic (e.g., a poorly written Regular Expression):** Attackers may be able to outsmart a weakness in the validation logic.

5.9 Application Areas:

- **Protocol Parsing:** When applications need to process fixed and highly structured data, including network packet parsing and abnormal detection, which is an unexpected format anywhere in the data, can also be very important.
- **User-Facing Web Forms:** Your front line of defense for login, search, and profile updates.

5.10 Principle of Least Privilege

This requires running the Web application or Web server with a limited user account that has only sufficient privilege for its execution and no other privilege than that.

5.11 Advantages:

- **Reduced Blast Radius:** This works as a safeguard measure. In case the developer makes a mistake and fails to prevent injection, the attacker only gets the permissions of that limited user, thereby failing to take control of the entire operating system.
- **Dual Defense Mechanism:** It works even when the vulnerability is not yet discovered (zero days).

5.12 Disadvantages:

- **Does Not Eliminate the Problem:** It does not prevent the injection from occurring but rather lessens the impact. The intruder is still able to potentially access or erase the data belonging to that restricted user account.
- **Setup Difficulty:** Is much more involved than setting up the entire system as root/administrator.

5.13 Application Areas:

- **Server Architecture and Containerization:** The universal best practice when deploying websites, databases, and microservices. Technologies such as Docker always operate under this concept because applications must be separated from the host operating system.

5.14 Evaluation of Server-Side Template Injection Mitigation Data Code Separation

The method used is referred to as **Data Code Separation using Context Variables**. By using a user's input in the form of a variable name `_var=user_input`, we ensure that the template engine will consider the input to be only a piece of data.

5.15 Advantages:

- **Addressing the Real Issue:** This method fully addresses vulnerability by design, since the template engine analyzes the structure of the template before evaluating any variable inputs. It is simply not possible that the user-supplied input will escape evaluation and become code.
- **No Dependencies Required:** This is a built-in function of nearly all modern template engines, from Jinja2, Twig, FreeMarker, Handlebars, and many others. There is no third-party software needed to apply it.
- **Prevents Secondary Vulnerabilities (like XSS):** Most modern template engines, including Jinja2, enforce automatic HTML-escaping on context variables. This means that if a user input `<script>alert(1)</script>`, it will be rendered as harmless text (`<script>...`), seamlessly preventing Cross-Site Scripting (XSS) alongside SSTI.
- **Negligible Performance Impact:** Passing variables to a template engine is the highly optimized, standard way these engines are designed to operate. It introduces no measurable performance overhead compared to raw string concatenation.

5.16 Disadvantages:

- **Possible Human Mistakes:** One of the biggest drawbacks is that it depends on developer discipline. Even one mistake from the developers, such as using the f-string or concatenation (using +) instead of a context variable makes the system susceptible again. This needs strict code reviewing.
- **Limited Flexibility:** For certain architecture of applications—Content Management Systems (CMS), for example—the administrator needs to have the freedom to edit the structure of the template itself. Under such circumstances, restricting the input to be just treated as data will defeat the purpose.
- **The "| safe" Filter Drawback:** Although this method makes sure that SSTI cannot happen at all, programmers tend to get frustrated when they cannot render HTML because of HTML escaping done automatically. This leads to them using filters such as "| safe" to do away with it.

5.17 Application Areas:

- **Web Application:** Any web application that uses frameworks like Flask, Django, Ruby on Rails, and Spring Boot to render dynamic HTML pages for the user.
- **Automated Email Applications:** Applications that send emails to the customer such as marketing emails, password reset emails, or receipt emails. In most cases, these applications use template engines to inject the user's name and other information to the body of the pre-defined HTML email.
- **Document Generator Applications:** Applications that create dynamic documents such as PDFs, invoices, and reports (for instance, generating invoices by inputting customer information into a template).
- **Infrastructure as Code:** Automated configuration management tools like Ansible, Helm, or Terraform use Jinja2 template engine or any other template engine to generate dynamic configuration files based on environment variables.

6 Conclusion

In this coursework, these injection vulnerabilities in web applications were explored, namely SQL Injection, OS Command Injection and Server-Side Template Injection. The report explored the sources of these vulnerabilities and what measures can be taken to mitigate them through background research, live demonstration, and actual mitigation testing. All three types of attack have a common cause, namely the lack of a separation between user-supplied data and executable code, which was found through background research.

This was a demonstration in practice with the demonstration. The SQL Injection attack was able to circumvent the authentication mechanism of the vulnerable PHP login form by altering the WHERE clause logic in it. Through the combination of un-sanitized string input and the use of the shell=True in the Flask-based ping tool, it was possible to inject arbitrary Linux commands through the use of a crafted input parameter. Unsanitised string input in combination with shell=True in the Flask-based ping tool enabled the injection of arbitrary Linux commands. In this attack, SSTI could be used against a Jinja2-powered Flask application, allowing for the evaluation of expressions server-side and the leaking of Flask configuration information via a URL parameter. The mitigation testing included a process that proved that there are reliable technical solutions for each vulnerability. All three OS Command Injection payloads were blocked by using a parameterized command list instead of shell=True and strict IP validation. The input into the Jinja2 template was changed to be a safe-named variable to remove the SSTI vulnerability without impacting the application.

The evaluation confirmed that no single control is effective alone and that a defense-in-depth approach using input validation, parameterization, and secure coding standards is the most effective means to provide secure protection. Although there are solutions available, injection vulnerabilities are still prevalent, signifying a lack of awareness and understanding of security in consistent application to development practice. Development, Security culture, and including security testing in the development lifecycle are still key elements to closing this gap and moving towards smarter web apps.

7 References

Abhijeet, 2025. *geeksforgeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/python/getting-started-with-jinja-template/>

[Accessed 14 May 2026].

Ansari, M. A., 2025. *How to prevent NoSQL injection attacks with JSON payloads*. [Online]

Available at: https://www.linkedin.com/posts/asadsec2723_nosql-injection-a-simple-guide-activity-7384911537779462144-nQfX

[Accessed May 2025].

Beschokov, M., 2025. *What is a Command Injection?*. [Online]

Available at: <https://www.wallarm.com/what/command-injection-definition>

Buenning, M., 2024. *ninjaone*. [Online]

Available at: <https://www.ninjaone.com/it-hub/endpoint-security/what-is-input-sanitization/>

[Accessed 9 May 2026].

cloudflare, 2026. *cloudflare*. [Online]

Available at: <https://www.cloudflare.com/learning/security/threats/sql-injection/>

[Accessed 15 May 2026].

Crespo-Martinez, E., 2020. *Vulnerability analysis with SQLMAP*. [Online]

Available at: <https://ingenius.ups.edu.ec/ingenius/article/view/25.2021.10>

Cristian-Alexandru Staicu, M. P. B. L., 2018. *Synode: Understanding and Automatically Preventing Injection Attack on Node.js*, Darmstadt, Germany: TU Darmstadt and Imperial College London.

Cybersecurity & Infrastructure Security Agency, 2024. *cisa.gov*. [Online]

Available at: <https://www.cisa.gov/resources-tools/resources/secure-design-alert-eliminating-os-command-injection-vulnerabilities>

[Accessed 15 May 2026].

Demir, B., 2020. *A Pentester's Guide to Server Side Template Injection (SSTI)*.

[Online]

Available at: <https://www.cobalt.io/blog/a-pentesters-guide-to-server-side-template-injection-ssti>

[Accessed May 2025].

Geeks, G. f., 2025. *SQL Injection*. [Online]

Available at: <https://www.geeksforgeeks.org/sql/sql-injection/>

Halfond, W. G. J. & Orso, A., 2020. *AMNESIA: Analysis and Monitoring for NEutralizing SQL-Injection Attacks*, Atlanta, GA: Georgia Institute of Technology.

Hoang Viet Long, T. A. T. D. T. N. V. C. H. M. H. N. T. K. S., 2020. *An efficient algorithm and tool for detecting dangerous website vulnerabilities*. [Online]

Available at: <https://dl.acm.org/doi/abs/10.1504/ijwgs.2020.106128>

imperva, 2026. *imperva*. [Online]

Available at: <https://www.imperva.com/learn/application-security/rate-limiting/>

[Accessed 9 May 2026].

imperva, 2026. *imperva*. [Online]

Available at: <https://www.imperva.com/learn/application-security/nosql-injection/>

[Accessed 9 May 2026].

Indusface, 2020. *What is Command Injection?*. [Online]

Available at: <https://www.indusface.com/learning/what-is-command-injection/>

[Accessed May 2026].

Kant, A., 2025. *SQL Injection: All Concepts, All Payloads, All In One*. [Online]

Available at: <https://dev.to/deoxys/sql-injection-all-concepts-all-payloads-all-in-one-4ch5>

Kartik, 2026. *geeksforgeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/python/flask-tutorial/>

[Accessed 14 May 2026].

Mahesh, 2026. *Jsmon.sh*. [Online]

Available at: <https://blogs.jsmon.sh/what-is-server-side-template-injection-ssti-ways-to-exploit-examples-and-impact/>

[Accessed 15 May 2026].

Mathur, M., 2025. *fastly*. [Online]
Available at: <https://www.fastly.com/blog/back-to-basics-os-command-injection>
[Accessed 15 May 2026].

Nasareddin, M., Alkhamaiseh, A., Qasaimeh, M. & Al-Qassas, R., 2021. A systematic review of detection and prevention techniques of SQL injection attacks. *Information Security Journal: A Global Perspective*, 32(4), pp. 252-265.

Nidecki, T. A., 2026. *invicti*. [Online]
Available at: <https://www.invicti.com/learn/os-command-injection>
[Accessed 15 May 2026].

OWASP TOP 10, 2025. *OWASP TOP 10*. [Online]
Available at: https://owasp.org/Top10/2025/A05_2025-Injection/
[Accessed 12 April 2026].

owasp, 2026. *owasp*. [Online]
Available at: https://owasp.org/www-project-web-security-testing-guide/v41/4-Web_Application_Security_Testing/07-Input_Validation_Testing/18-Testing_for_Server_Side_Template_Injection
[Accessed 15 May 2026].

owasp, 2026. *owasp.org*. [Online]
Available at: https://owasp.org/www-community/vulnerabilities/CRLF_Injection
[Accessed 9 May 2026].

Paul, A., Sharma, V. & Olukoya, O., 2024. SQL injection attack: Detection, prioritization & prevention. *Journal of Information Security and Applications*, Volume 85, pp. 103871-103871.

portswigger, 2026. *Portswigger*. [Online]
Available at: <https://portswigger.net/web-security/os-command-injection>
[Accessed 9 May 2026].

PortSwigger, 2026. *SQL Injection*. [Online]
Available at: <https://portswigger.net/web-security/sql-injection>
[Accessed May 2026].

Saafan, A., 2026. *nilebits*. [Online]
Available at: https://www.nilebits.com/blog/2024/06/sql-injection-understanding-the-threat/?utm_source=Pinterest&utm_medium=organic
[Accessed 12 April 2026].

Sanghavi, K. & Samdani, K., 2024. Overview of Sql Injection Types, Defense Tools and Prevention Techniques. *Tuijin Jishu/Journal of Propulsion Technology*, 45(4), pp. 2780-2796.

Twingate Team, 2024. *Twingate Team*. [Online]
Available at: <https://www.twingate.com/blog/glossary/server-side%20template%20injection>
[Accessed 15 May 2026].

William G. J. Halfond, A. O., 2005. *AMNESIA: analysis and monitoring for NEutralizing SQL-injection attacks*. [Online]
Available at: <https://dl.acm.org/doi/10.1145/1101908.1101935>

8 Appendix

8.1 [Code for vulnerable Python Application](#)

```
from flask import Flask, request.
```

```
import subprocess.
```

```
app = Flask(__name__)
```

```
HTML_FORM = """
```

```
<h2>Network Diagnostic Tool</h2>
```

```
<p>Enter an IP address to ping:</p>
```

```
<form method="POST" action="/ping">
```

```
<input type="text" name="ip_address" placeholder="e.g., 127.0.0.1">
```

```
<button type="submit">Ping</button>
```

```
</form>
```

```
"""
```

```
@app.route('/')
```

```
def index():  
    return HTML_FORM  
  
@app.route('/ping', methods=['POST'])  
def ping_server():  
    user_input = request.form.get('ip_address')  
  
    # THE VULNERABILITY: Blindly trusting input in a shell command  
    command = "ping -c 1 " + user_input  
  
    try:  
        output = subprocess.check_output(command, shell=True, text=True,  
stderr=subprocess.STDOUT)  
        return f"<h3>Results:</h3><pre>{output}</pre>"  
    except subprocess.CalledProcessError as e:  
        return f"<h3>Error:</h3><pre>{e.output}</pre>"  
  
if __name__ == '__main__':  
    app.run(debug=True, port=5000)
```

8.2 Patched Code

```
from flask import Flask, request  
  
import subprocess  
  
import ipaddress # Added for strict input validation  
  
app = Flask(__name__)
```

```
HTML_FORM = """

    <h2>Network Diagnostic Tool (Secure Version)</h2>

    <p>Enter an IP address to ping:</p>

    <form method="POST" action="/ping">

        <input type="text" name="ip_address" placeholder="e.g., 127.0.0.1">

        <button type="submit">Ping</button>

    </form>

    """
```

```
@app.route('/')
def index():
    return HTML_FORM

@app.route('/ping', methods=['POST'])
def ping_server():
    user_input = request.form.get('ip_address', "").strip()

    # DEFENSE LAYER 1: Strict Input Validation
    try:
        # This will immediately fail if the input is not a perfectly formatted IP
        valid_ip = str(ipaddress.ip_address(user_input))
    except ValueError:
```

```
    return "<h3>Security Error:</h3><p>Invalid IP address format detected. Request  
dropped.</p>"
```

```
# DEFENSE LAYER 2: Parameterization
```

```
# The command is broken down into a list. The shell is not invoked.
```

```
command_list = ["ping", "-c", "1", valid_ip]
```

```
try:
```

```
    # shell=False ensures the OS treats the list items strictly as arguments
```

```
    output = subprocess.check_output(command_list, shell=False, text=True,  
stderr=subprocess.STDOUT)
```

```
    return f"<h3>Results:</h3><pre>{output}</pre>"
```

```
except subprocess.CalledProcessError as e:
```

```
    return f"<h3>Error:</h3><pre>{e.output}</pre>"
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True, port=5000)
```